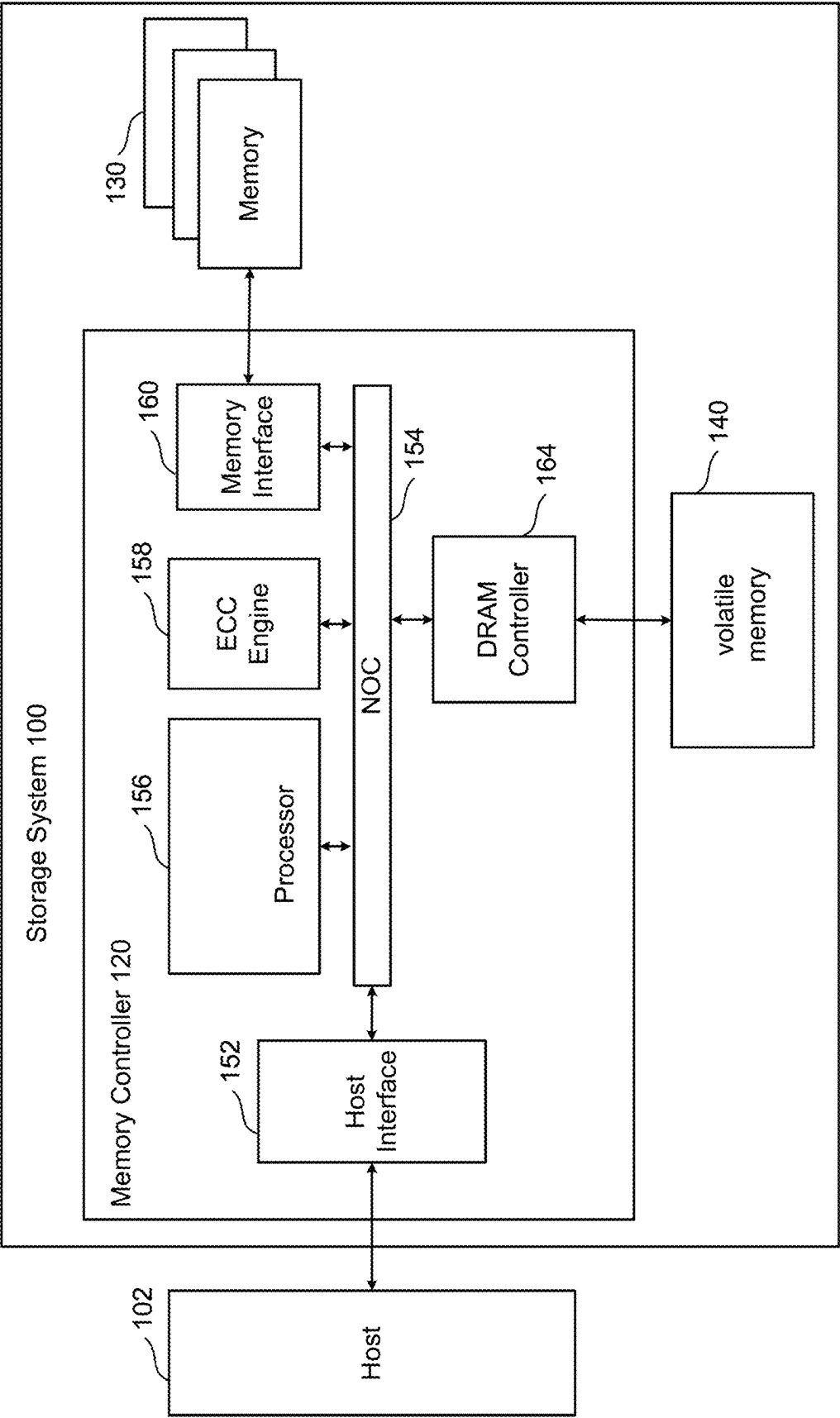
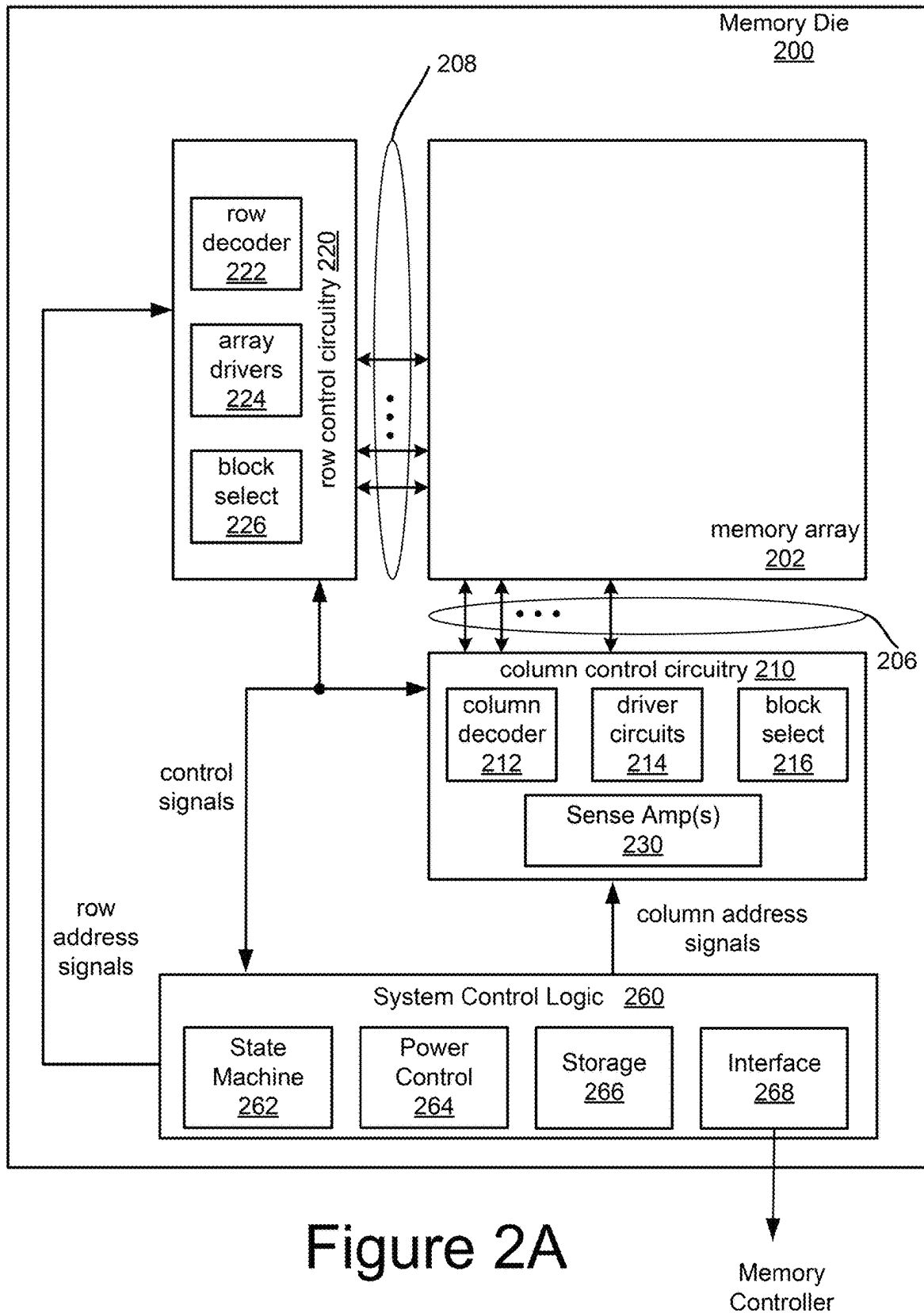


Figure 1





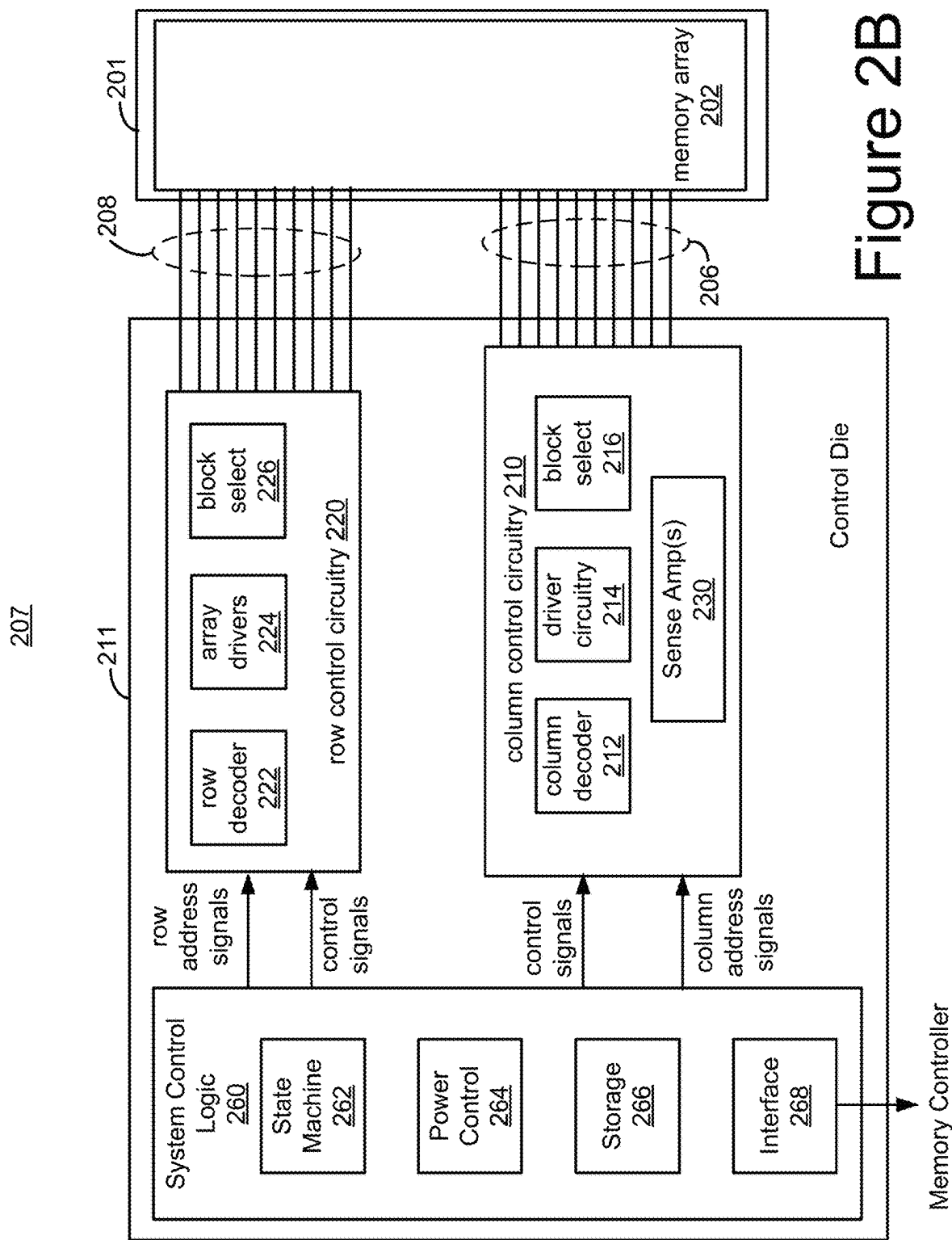


Figure 3A

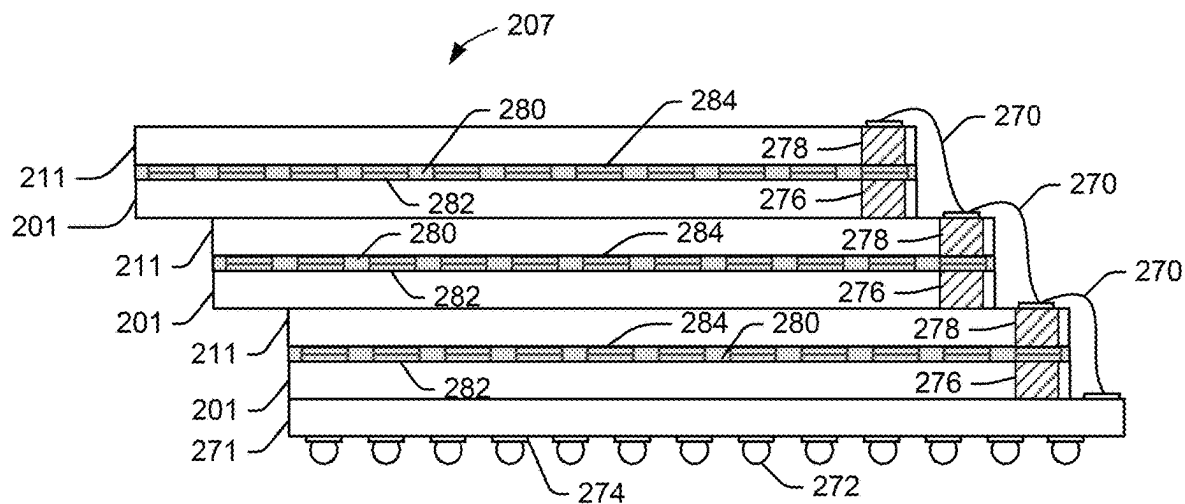


Figure 3B

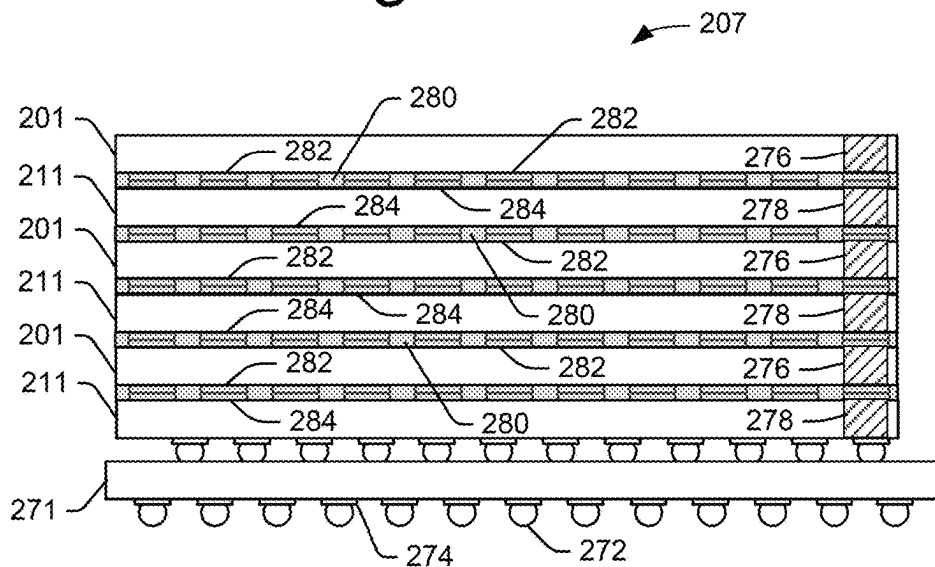


Figure 4

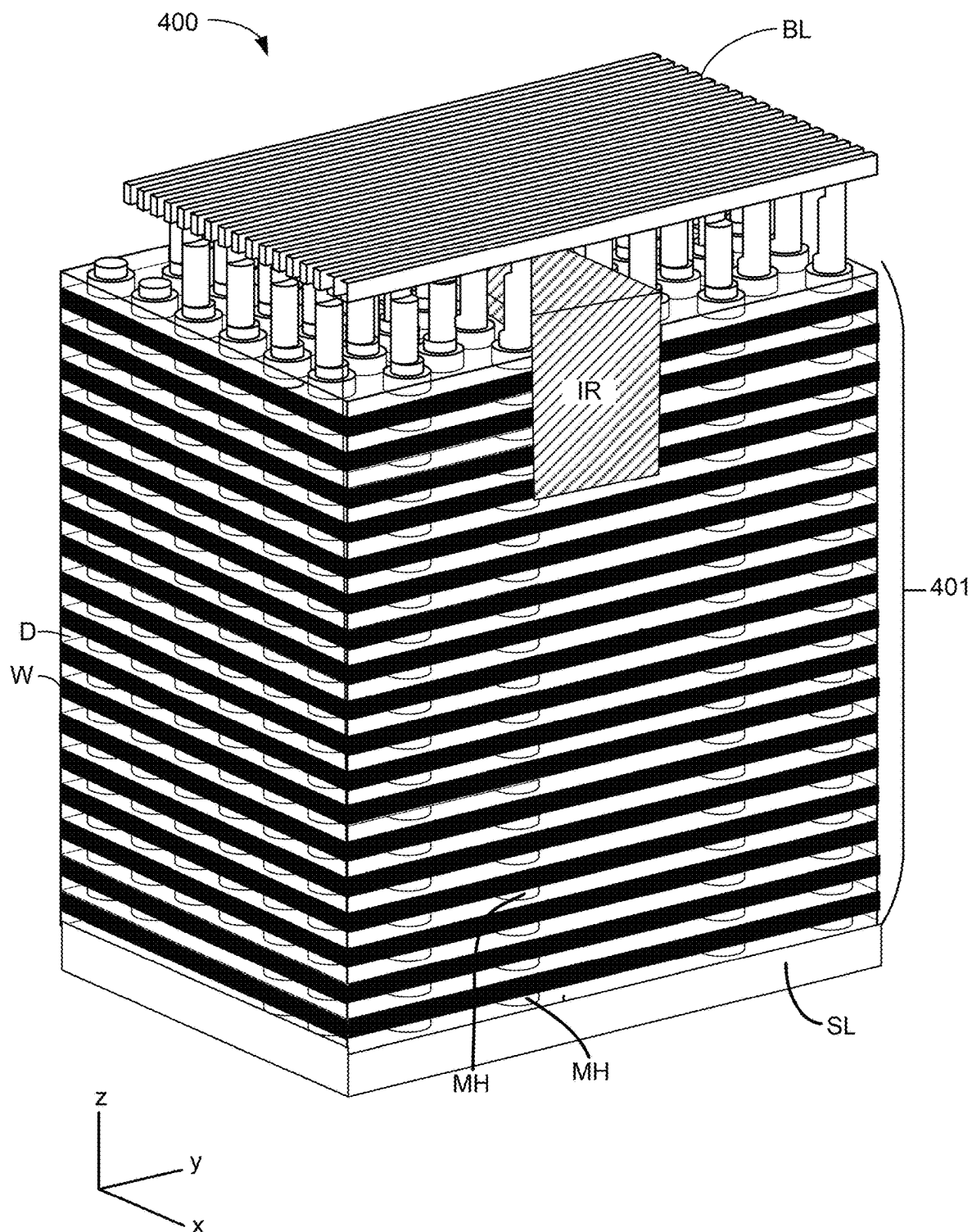


Figure 4A

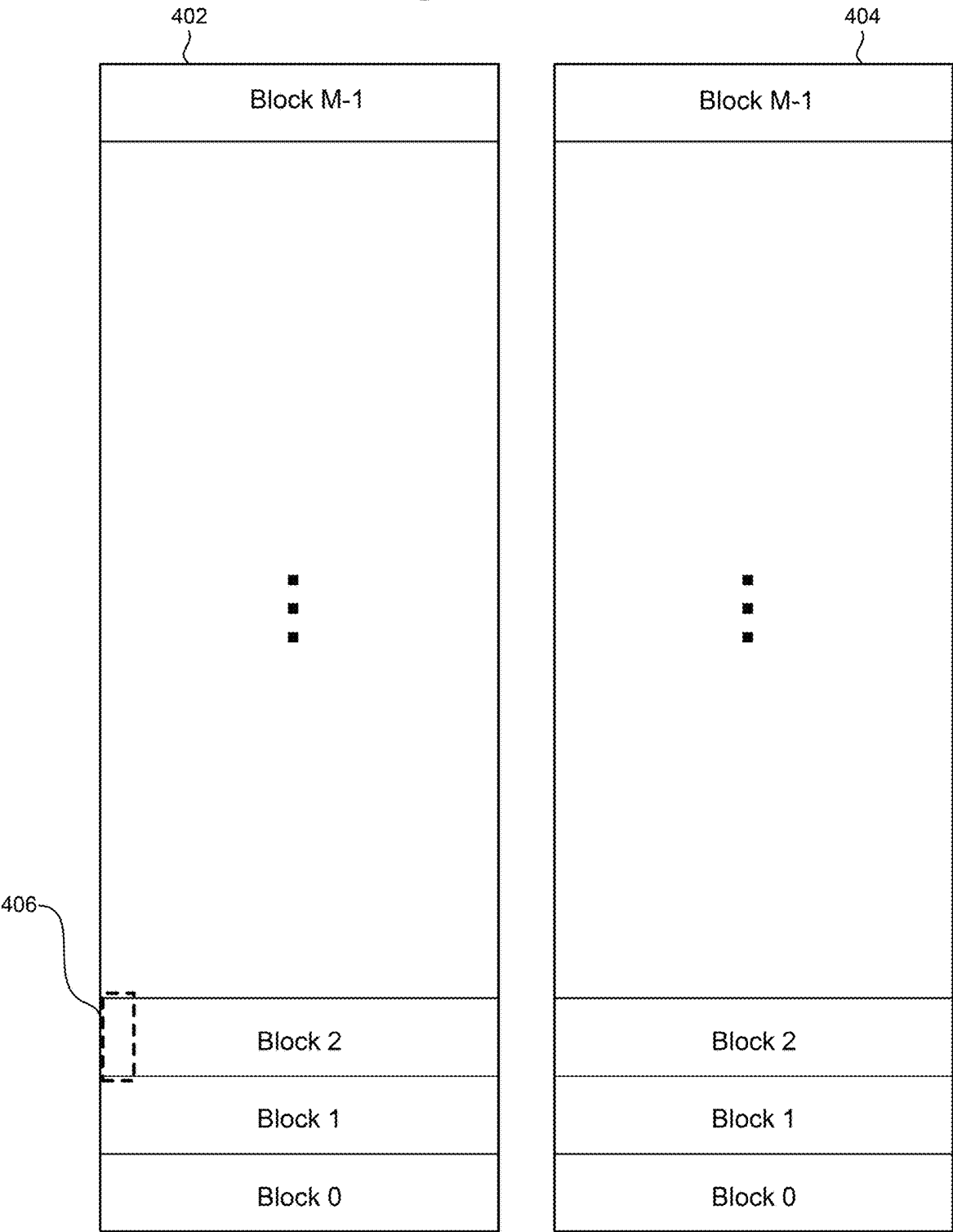


Figure 4B

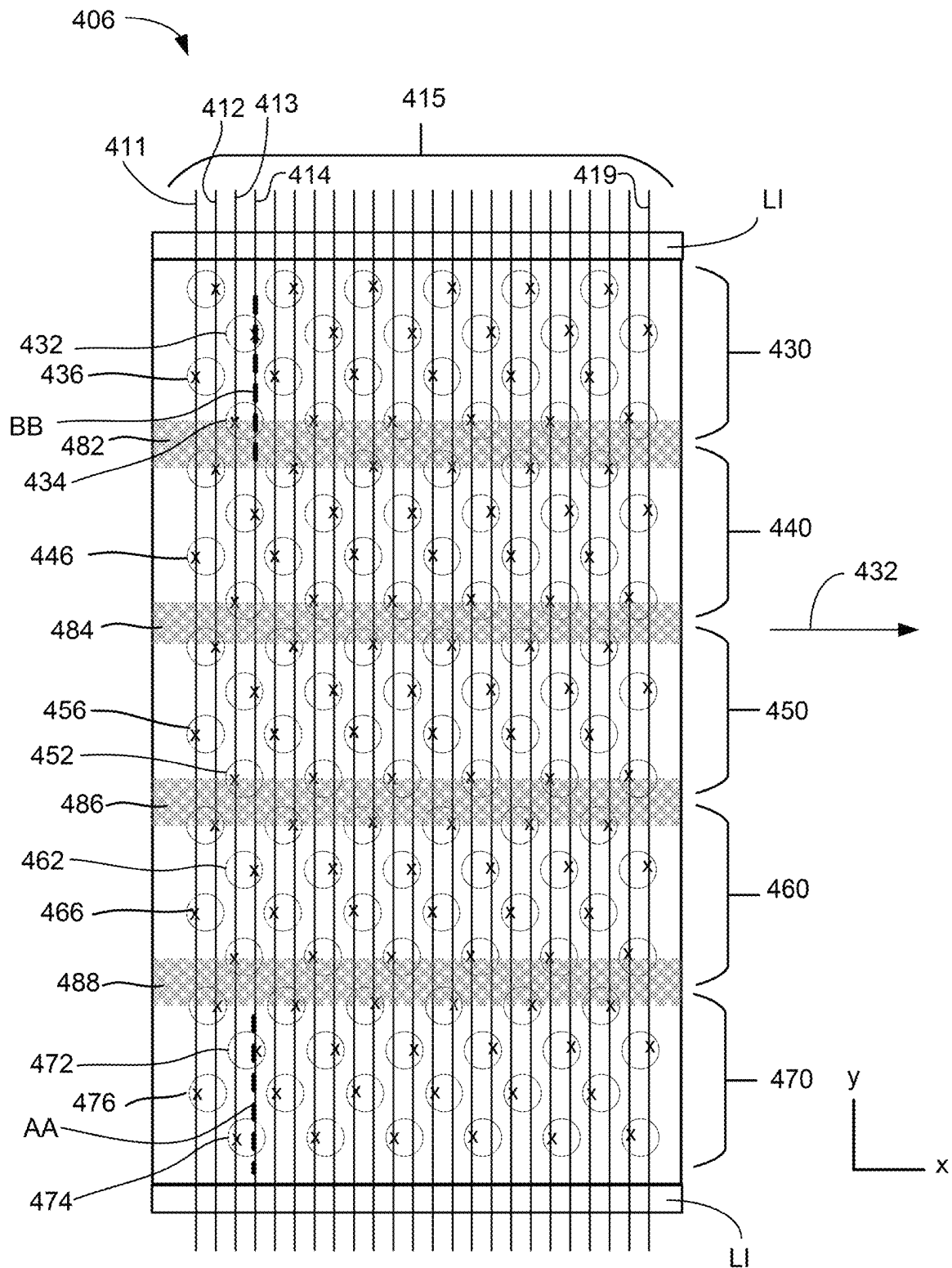


Figure 4C

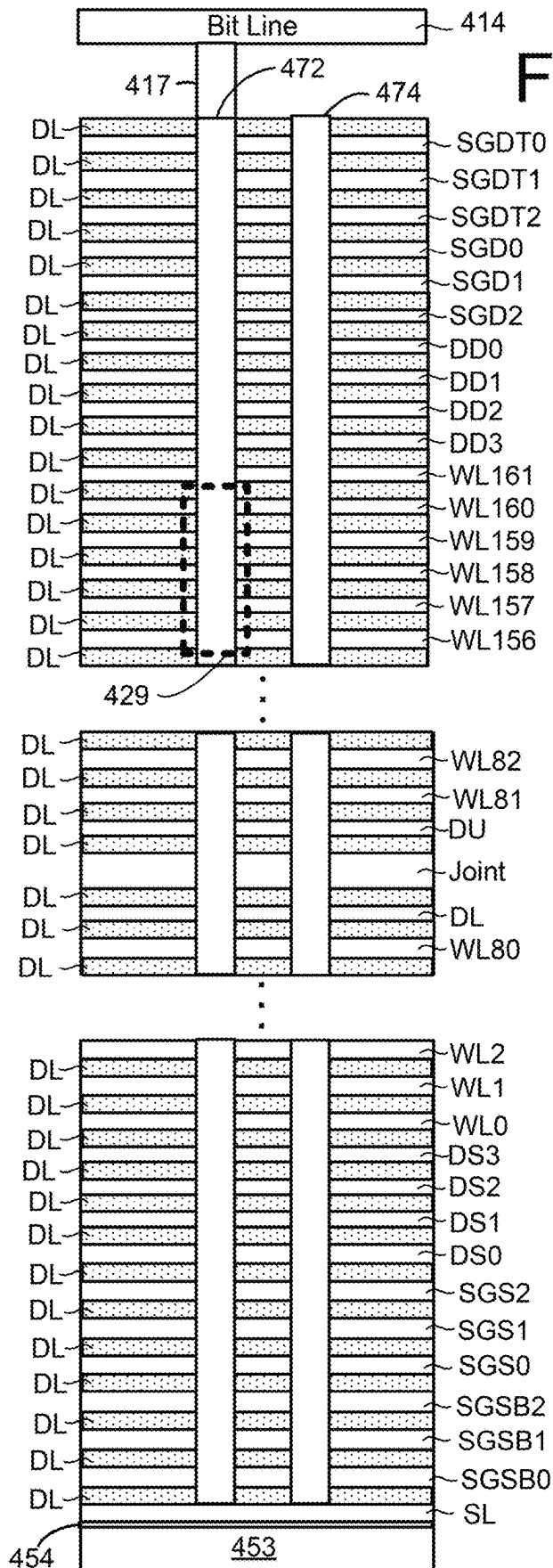
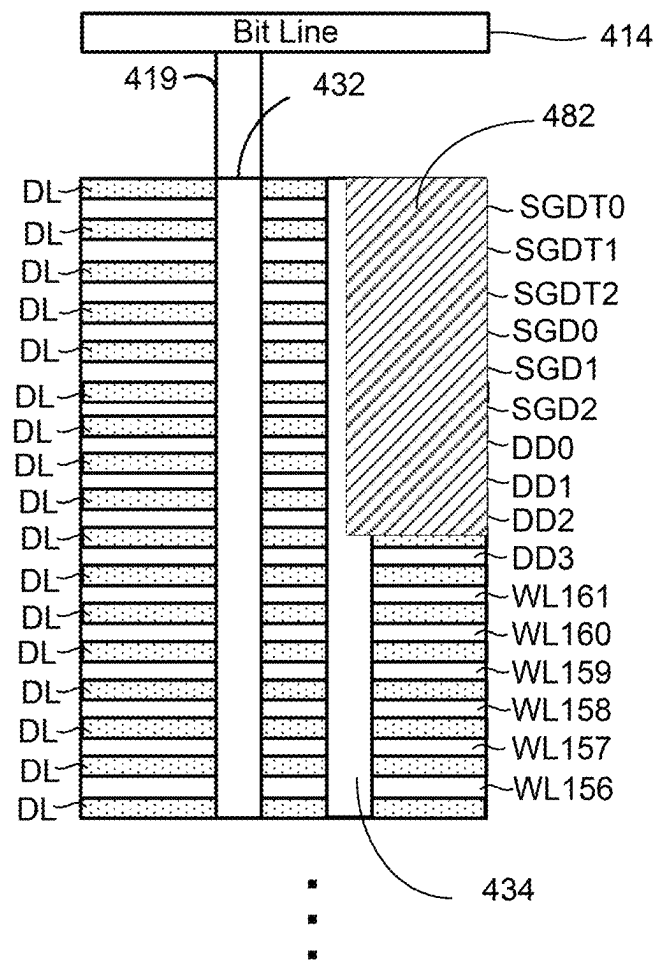


Figure 4D



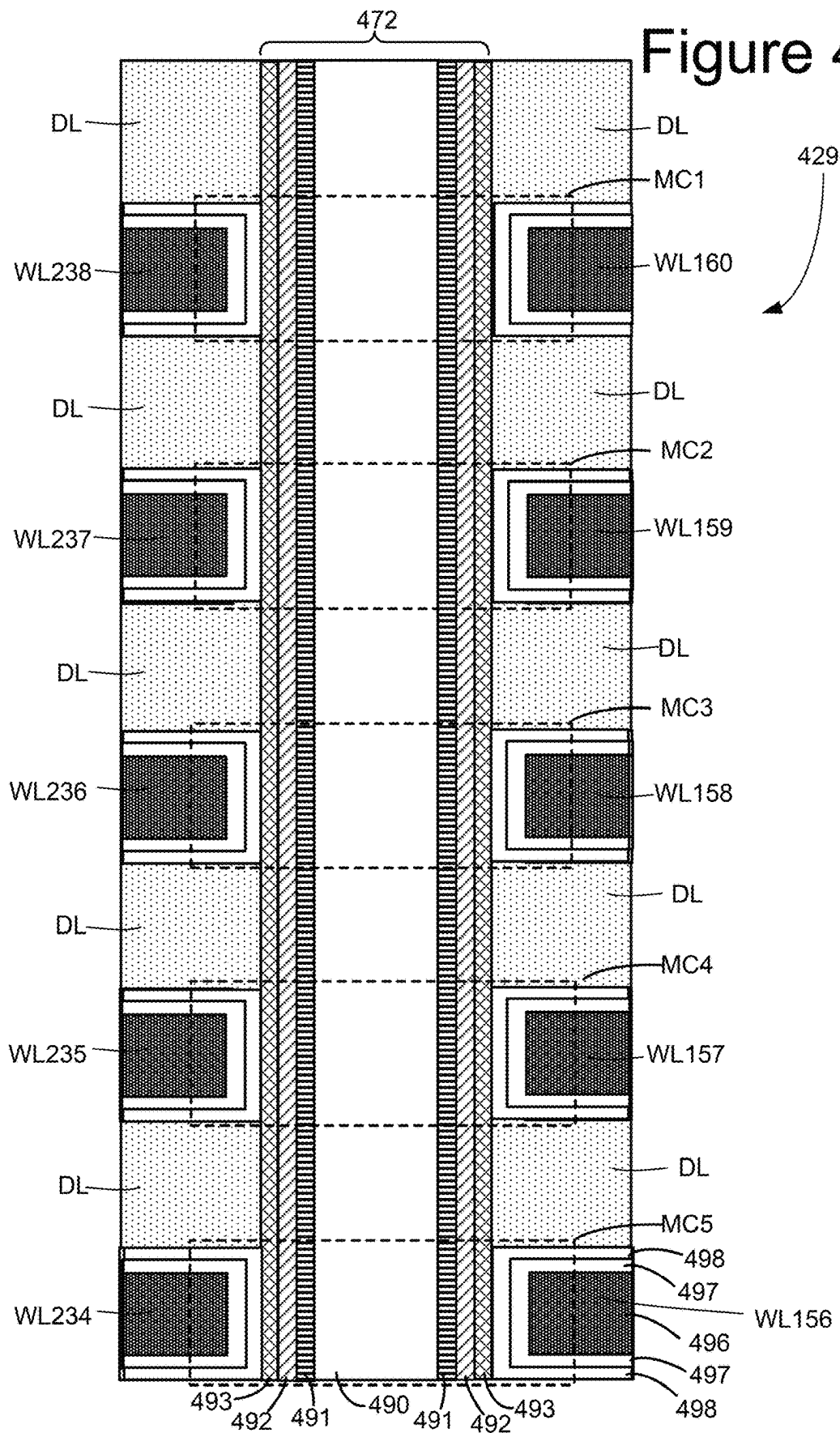


Figure 4F

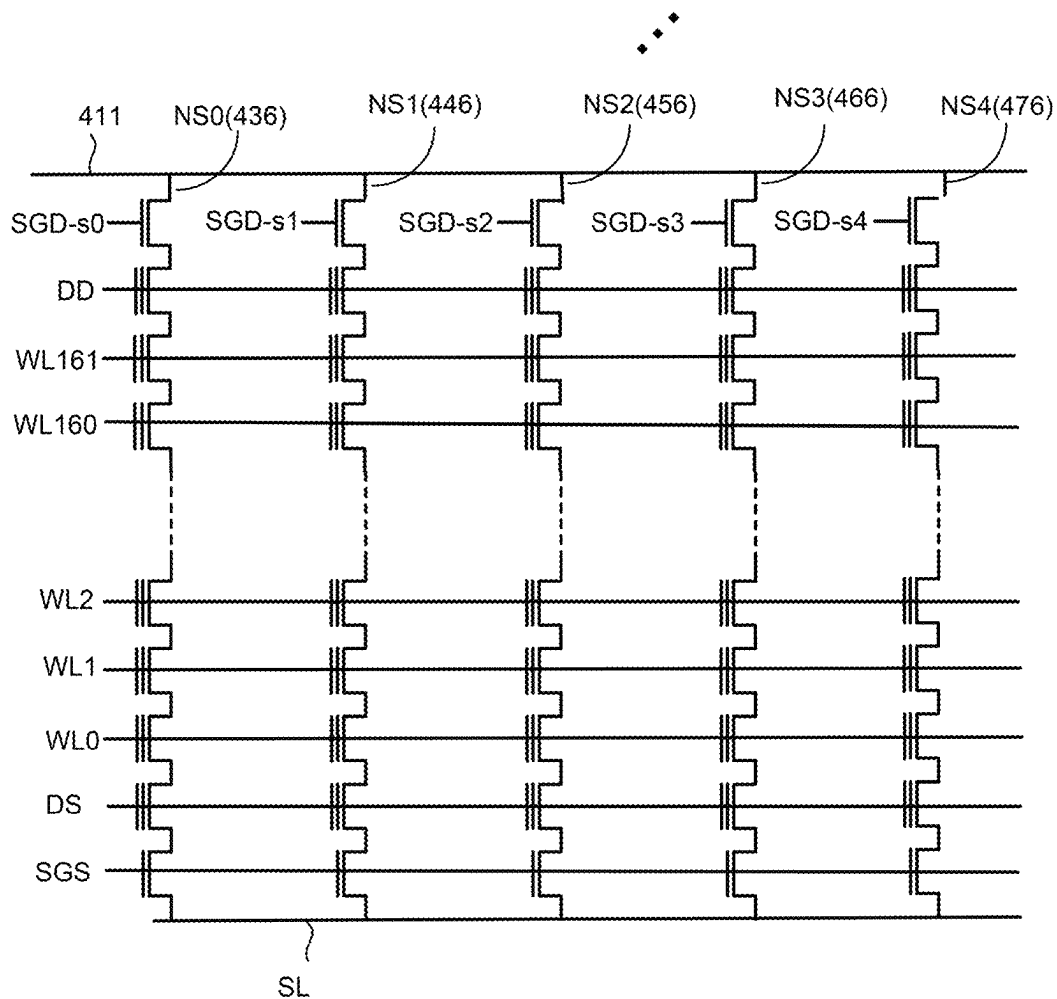


Figure 5A

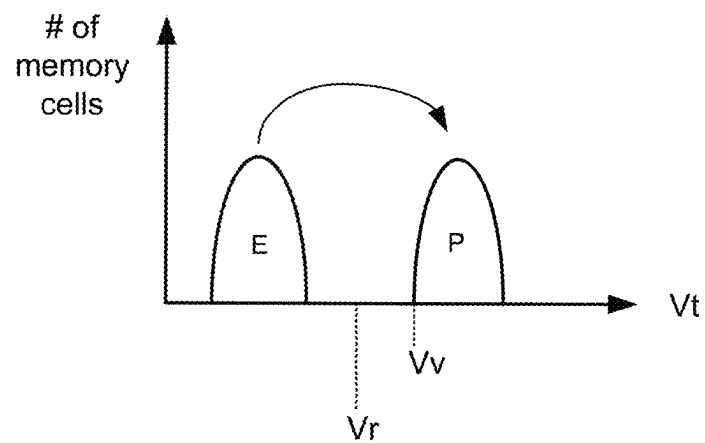


Figure 5B

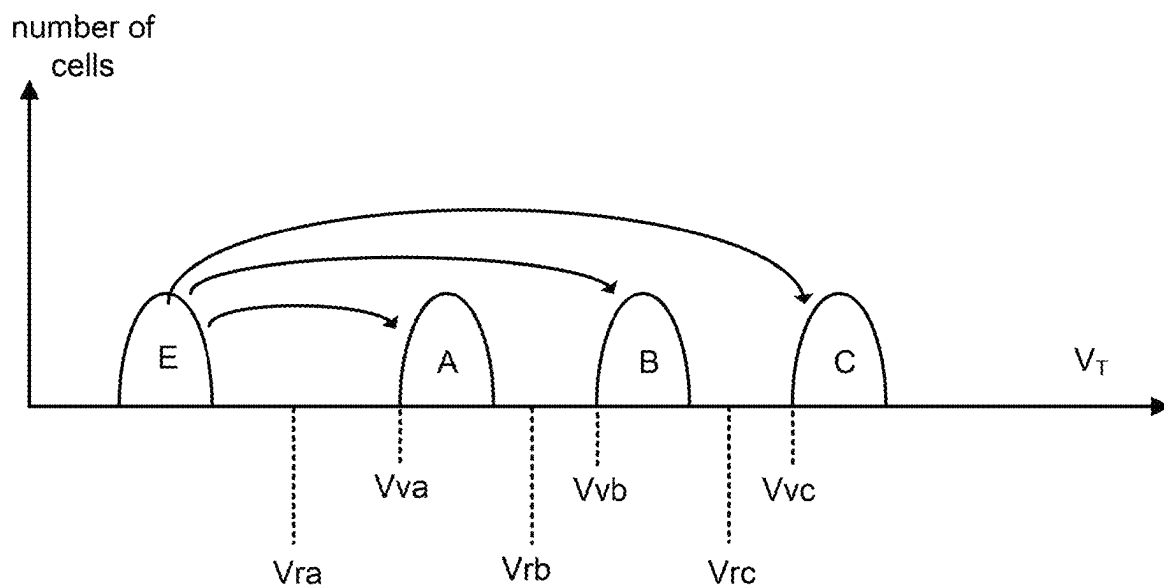


Figure 5C

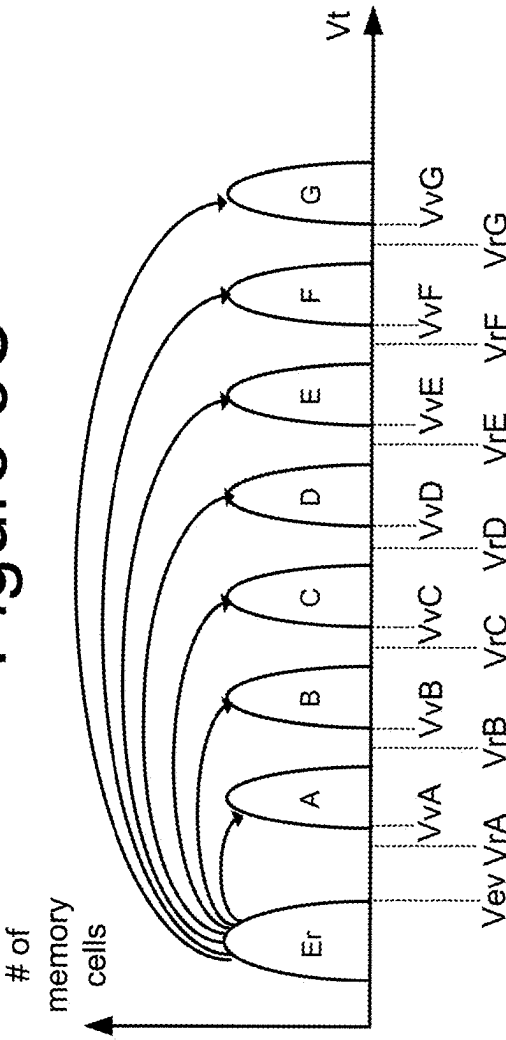


Figure 5D

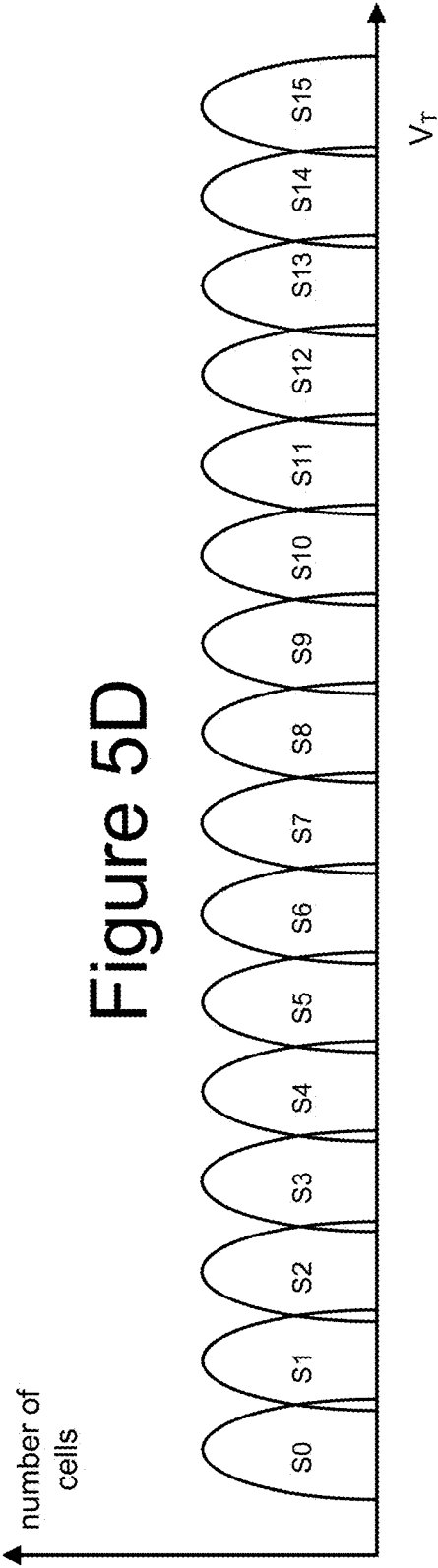


Figure 6

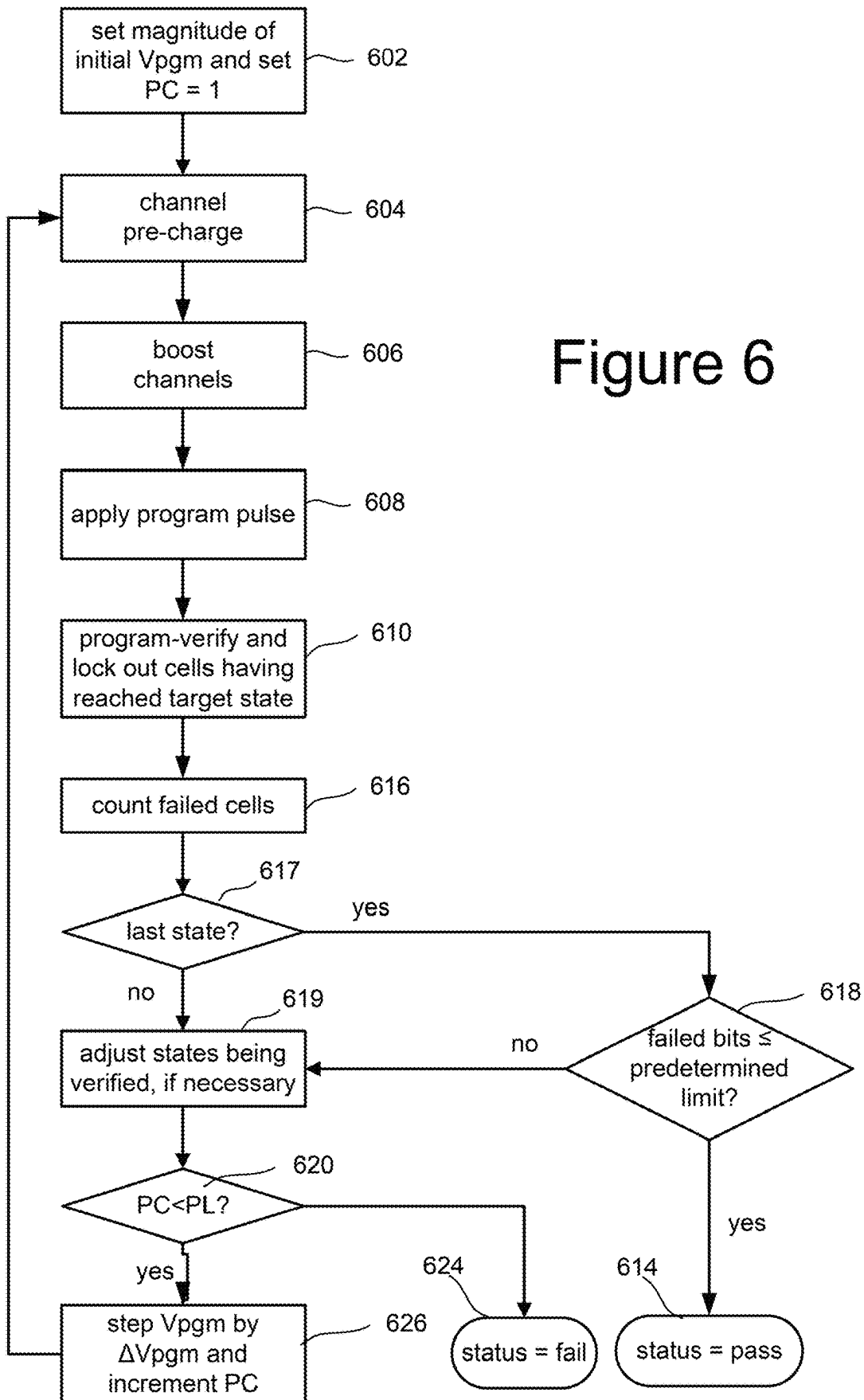


Figure 7A

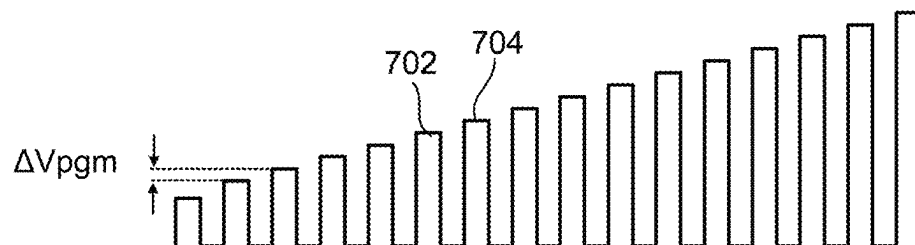


Figure 7B

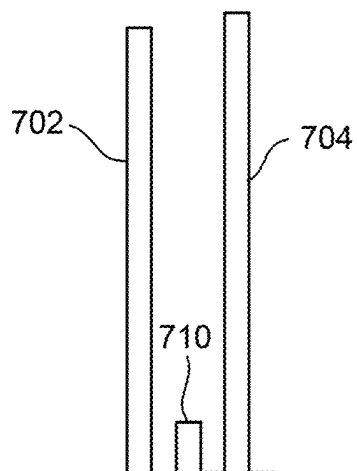


Figure 7C

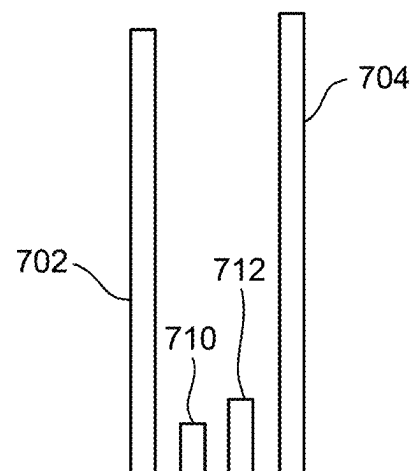


Figure 8A

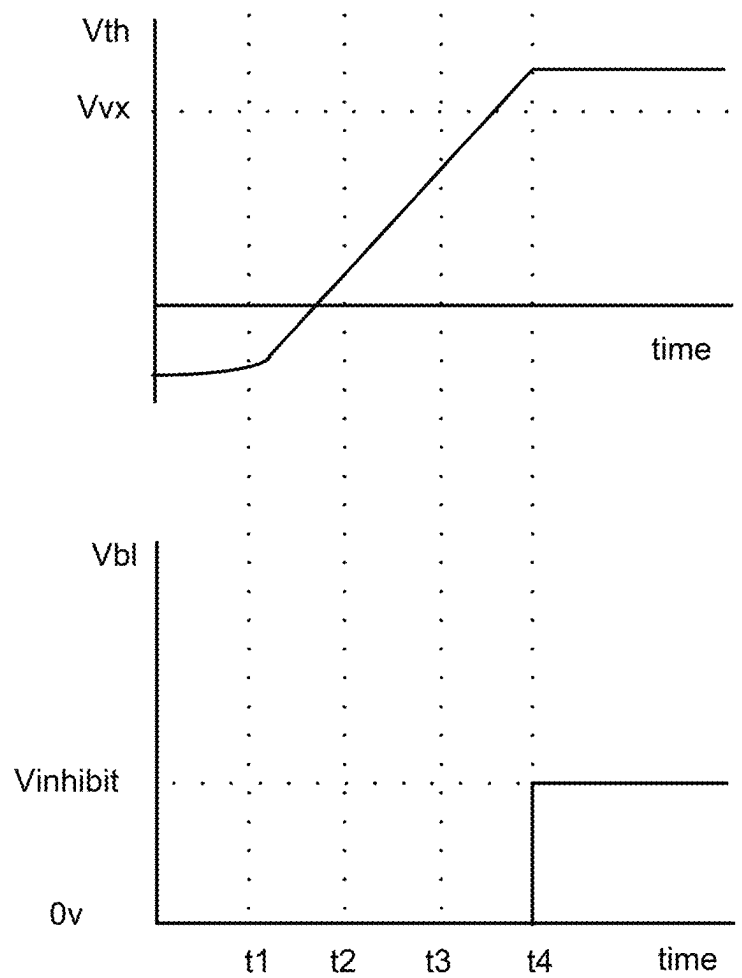


Figure 8B

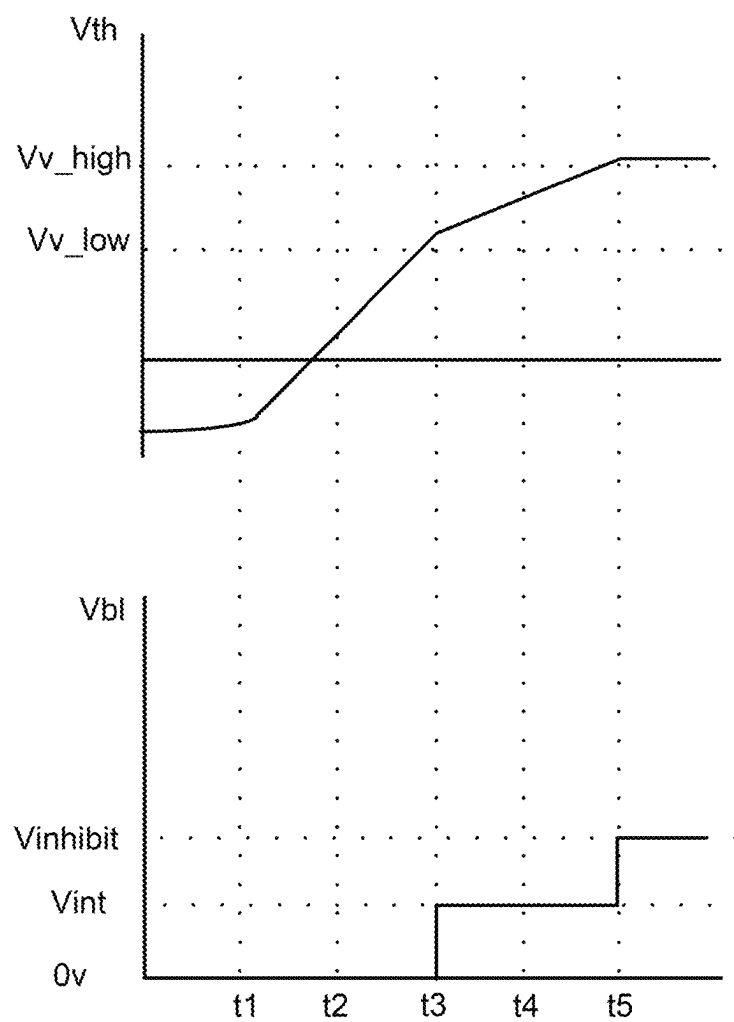


Figure 9

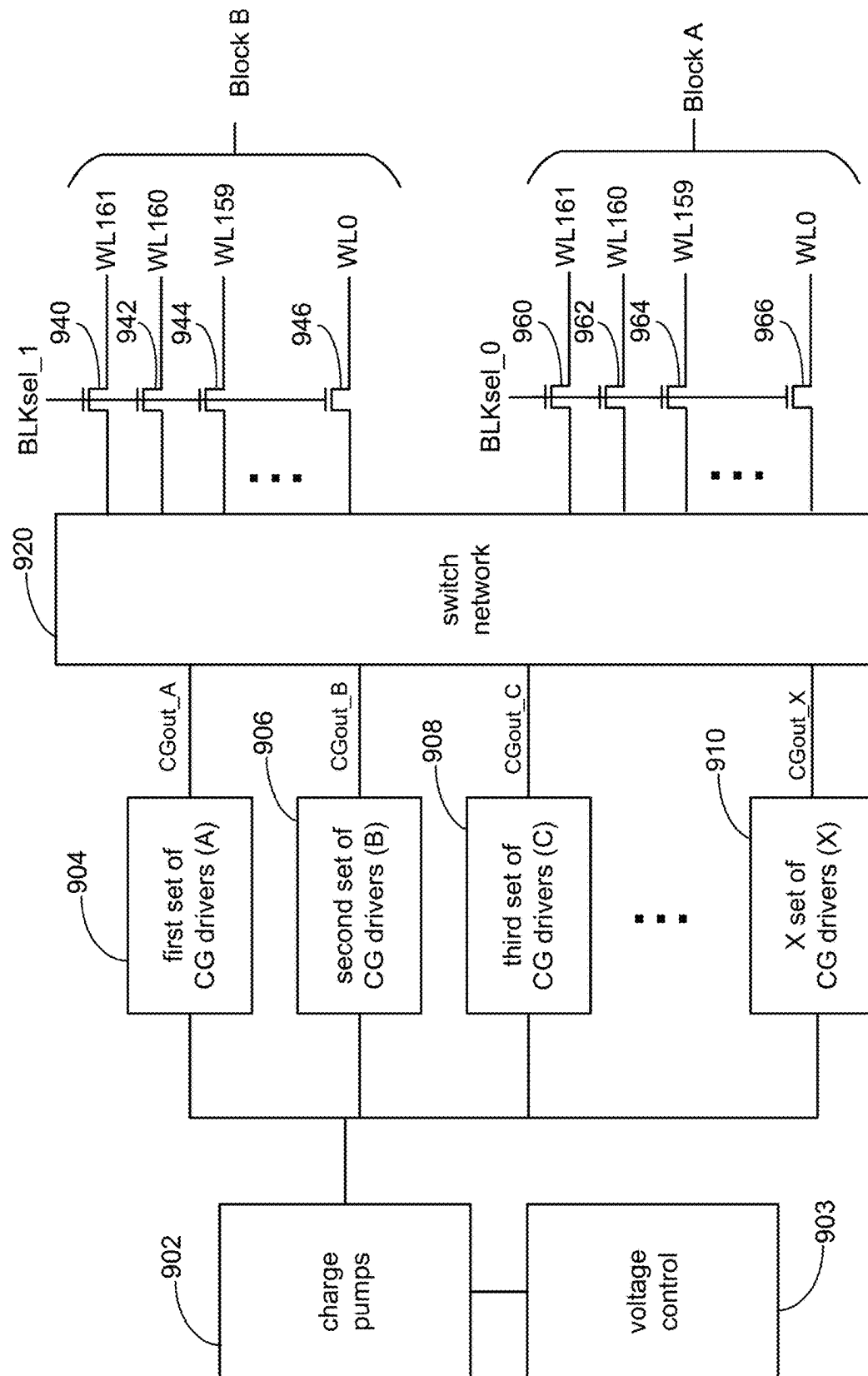


Figure 10

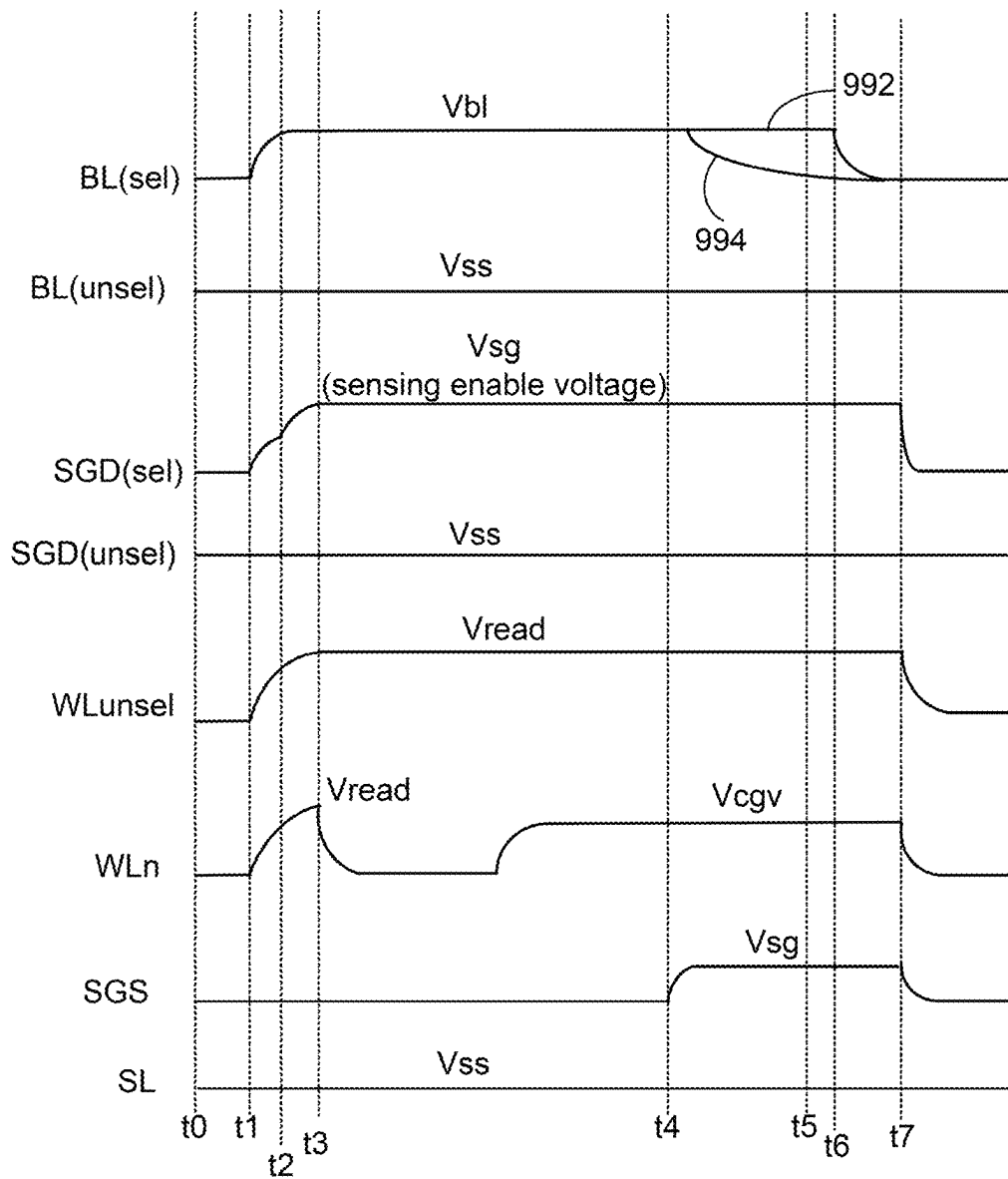


Figure 11

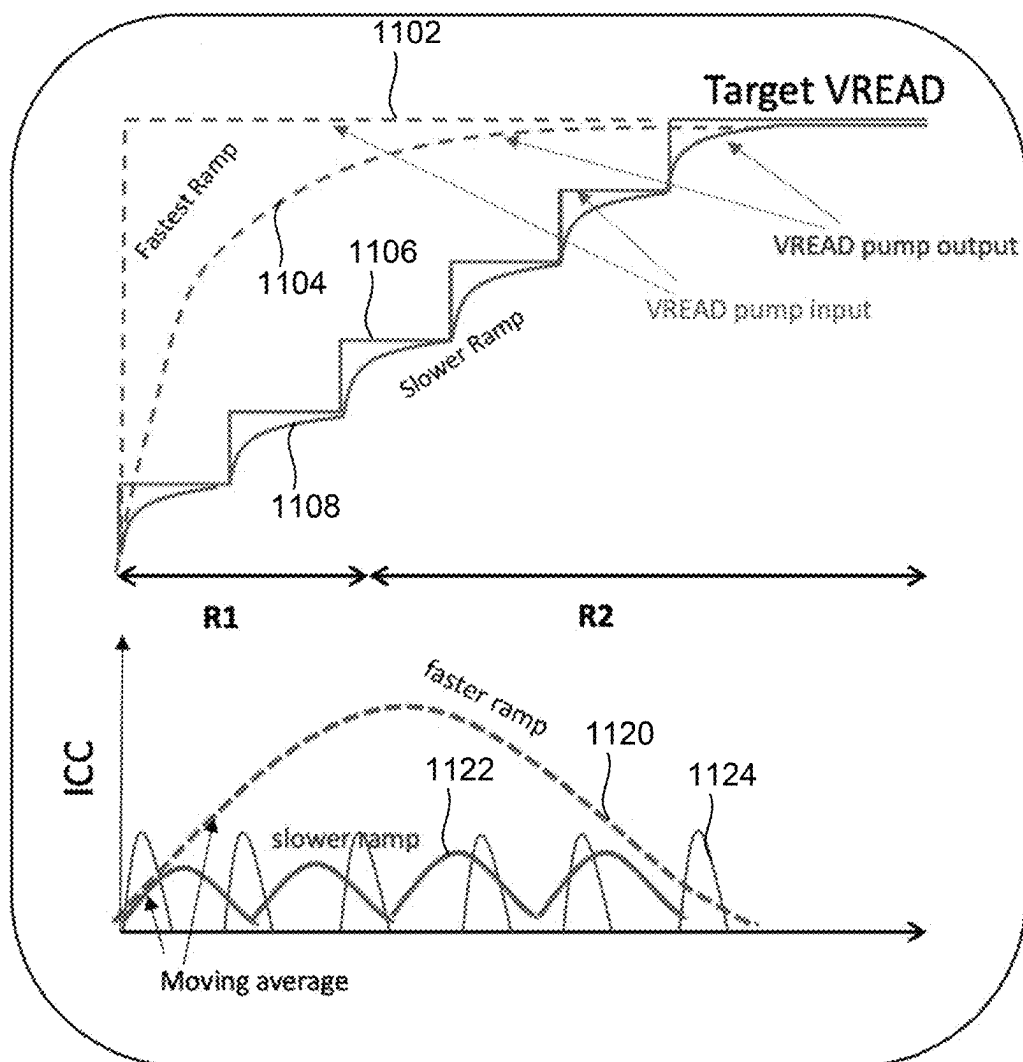


Figure 12

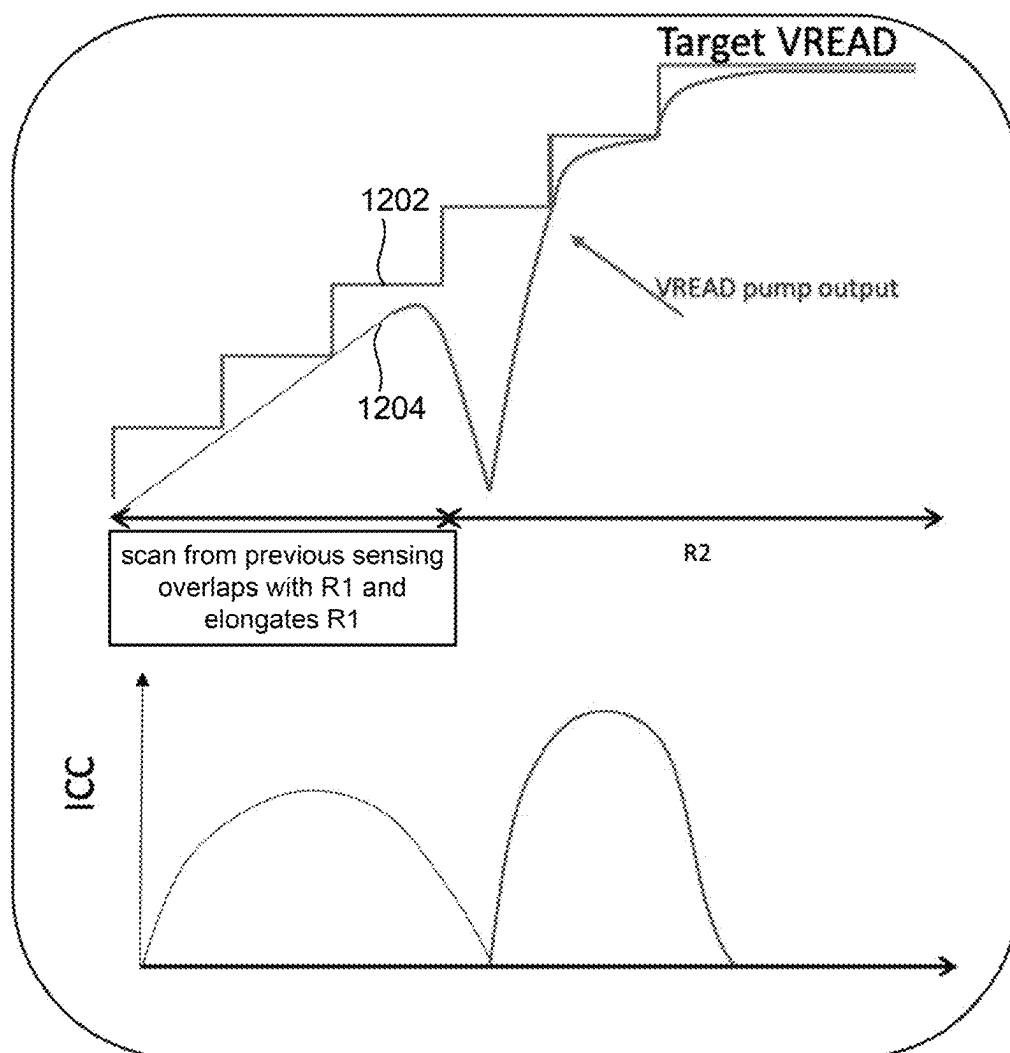


Figure 13

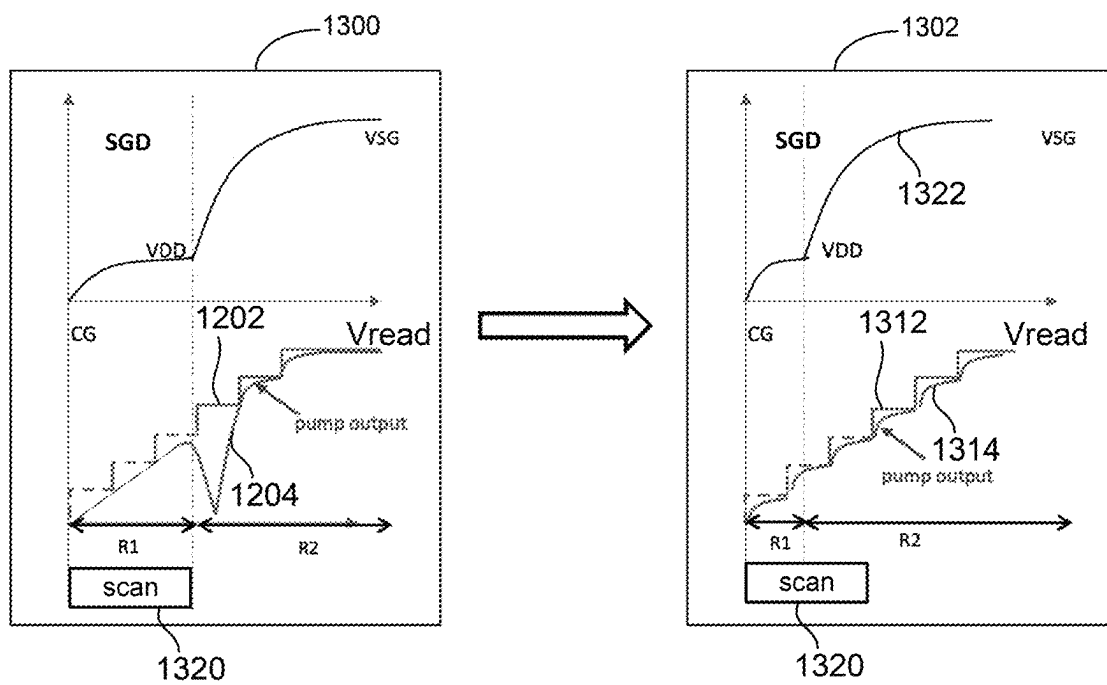


Figure 14

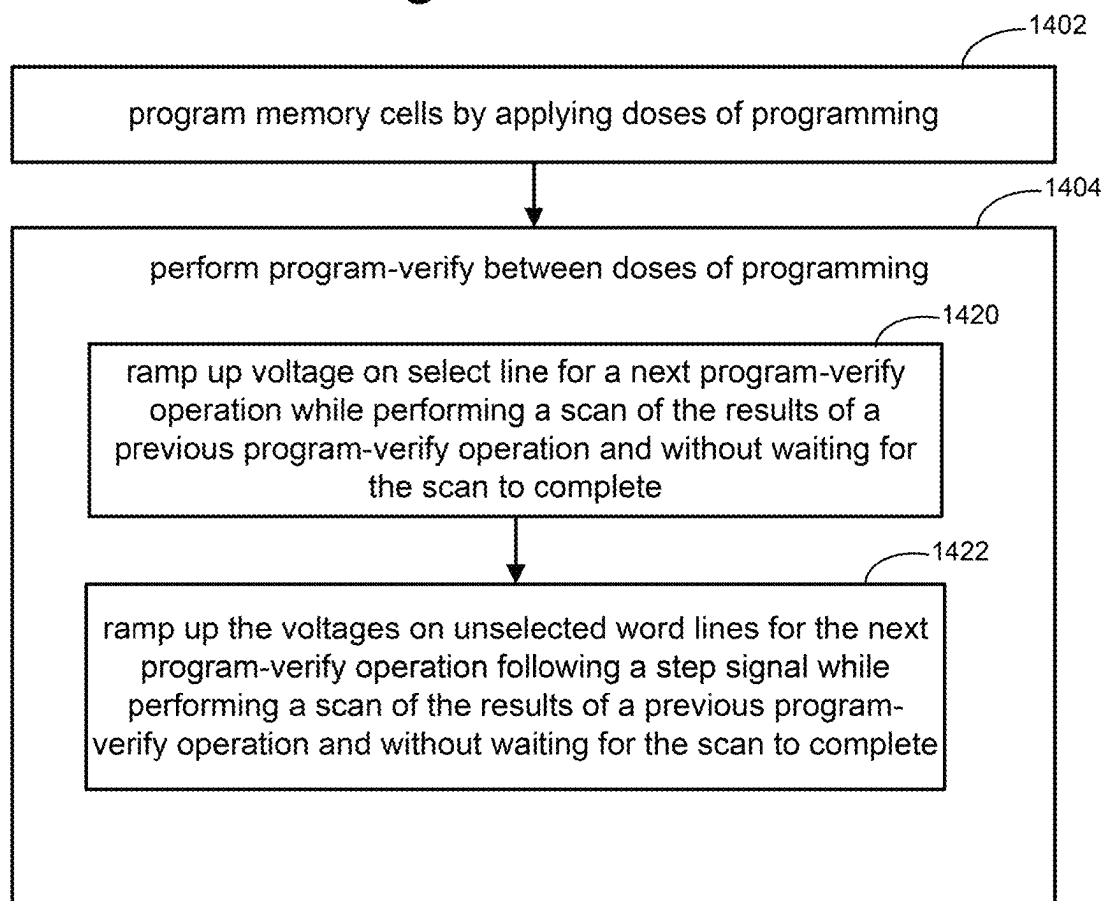


Figure 15

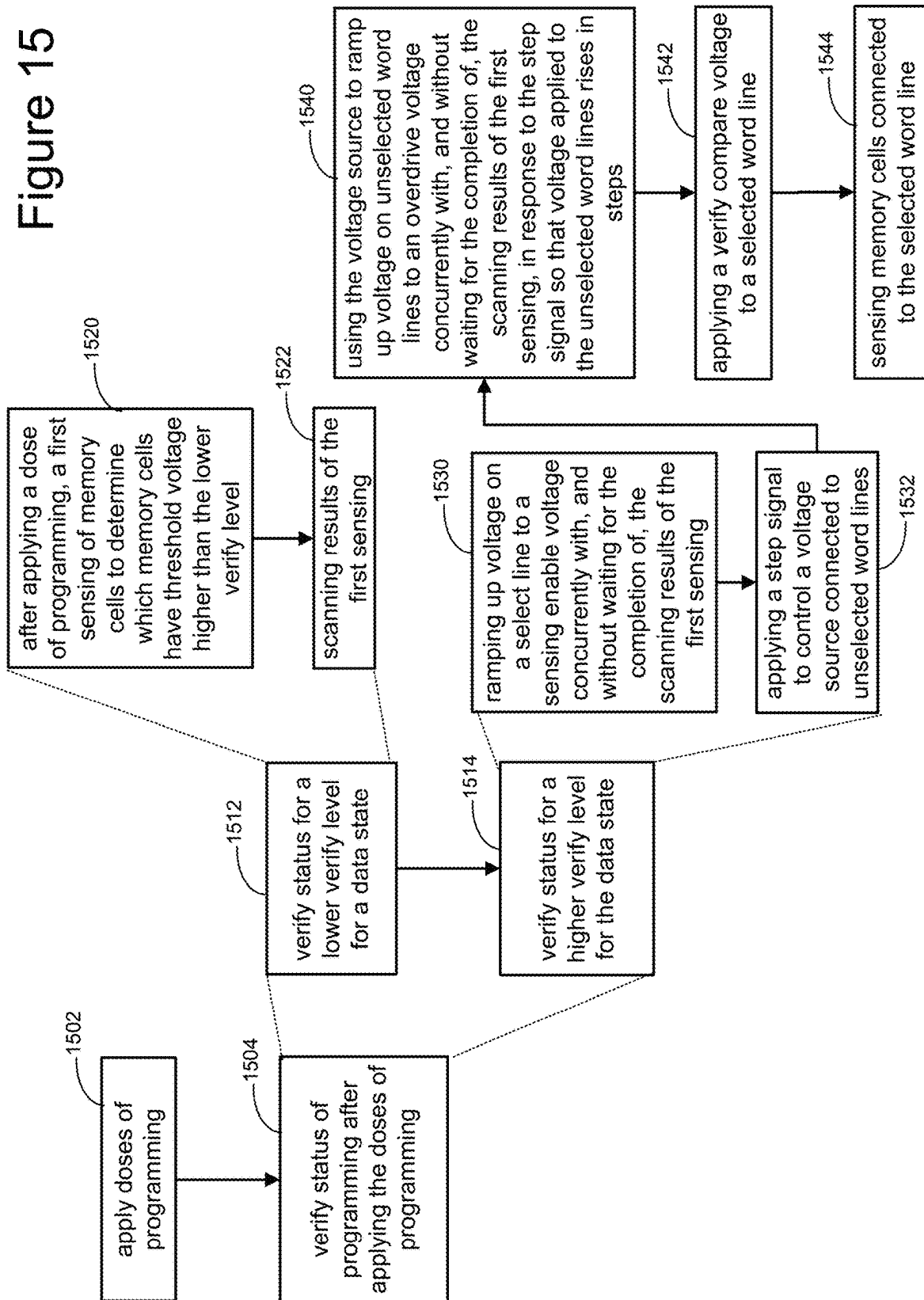


Figure 16

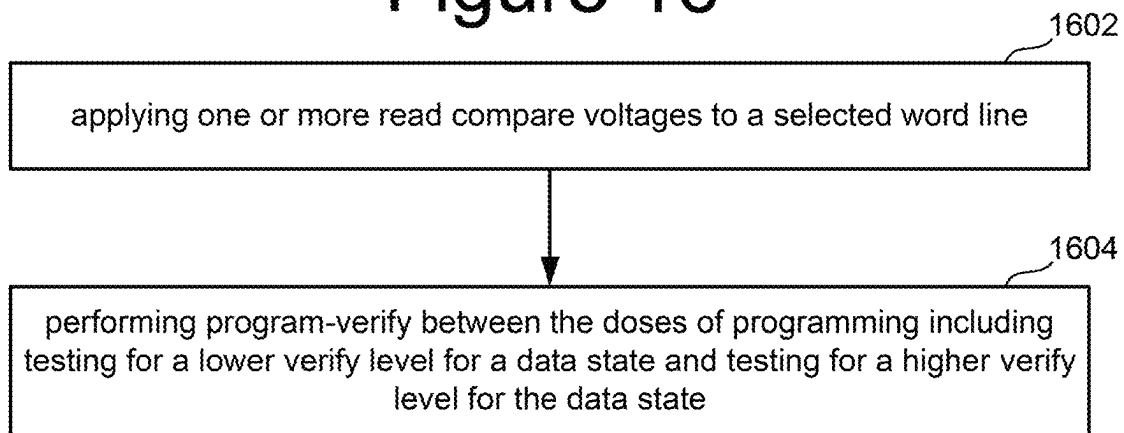
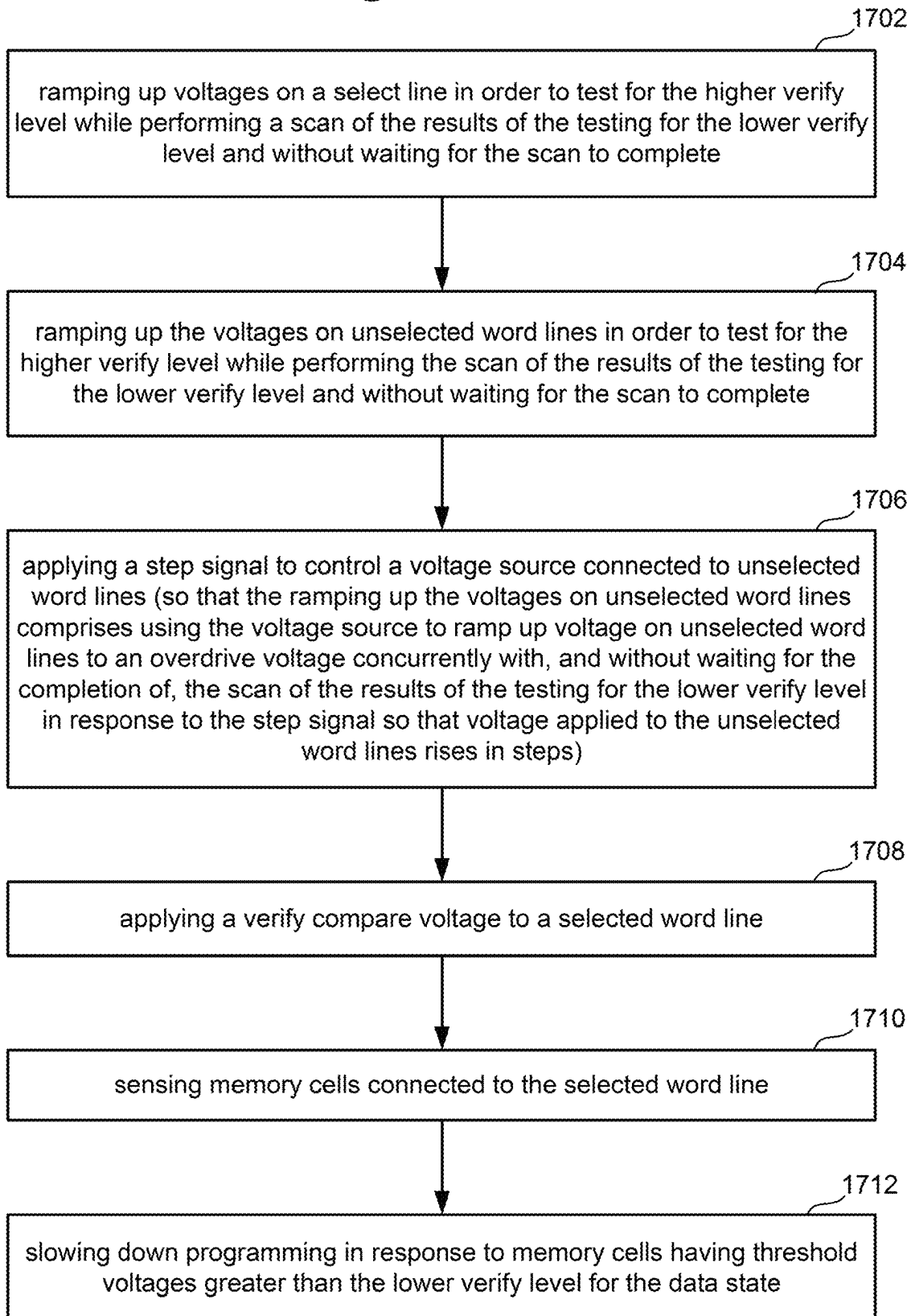


Figure 17



1

NON-VOLATILE MEMORY WITH LOWER CURRENT PROGRAM-VERIFY

CLAIM TO PRIORITY

The present application claims priority from U.S. Provisional Patent Application No. 63/419,210, entitled “NON-VOLATILE MEMORY WITH LOWER CURRENT PROGRAM-VERIFY”, filed Oct. 25, 2022, incorporated by reference herein in its entirety.

BACKGROUND

The present disclosure relates to non-volatile storage.

Semiconductor memory is widely used in various electronic devices such as cellular telephones, digital cameras, personal digital assistants, medical electronics, mobile computing devices, servers, solid state drives, non-mobile computing devices and other devices. Semiconductor memory may comprise non-volatile memory or volatile memory. Non-volatile memory allows information to be stored and retained even when the non-volatile memory is not connected to a source of power (e.g., a battery). One example of non-volatile memory is flash memory (e.g., NAND-type and NOR-type flash memory).

Many electronic devices that use non-volatile memory are mobile devices that rely on batteries for power. Therefore, it is advantageous for non-volatile memory to operate at lower power (including the use of lower currents) in order to most efficiently use the battery. Use of lower currents also helps to prevent overheating of the electronic devices.

BRIEF DESCRIPTION OF THE DRAWINGS

Like-numbered elements refer to common components in the different figures.

FIG. 1 is a block diagram depicting one embodiment of a storage system.

FIG. 2A is a block diagram of one embodiment of a memory die.

FIG. 2B is a block diagram of one embodiment of an integrated memory assembly.

FIGS. 3A and 3B depict different embodiments of integrated memory assemblies.

FIG. 4 is a perspective view of a portion of one embodiment of a monolithic three dimensional memory structure.

FIG. 4A is a block diagram of one embodiment of a memory structure having two planes.

FIG. 4B depicts a top view of a portion of one embodiment of a block of memory cells.

FIG. 4C depicts a cross sectional view of a portion of one embodiment of a block of memory cells.

FIG. 4D depicts a cross sectional view of a portion of one embodiment of a block of memory cells.

FIG. 4E is a cross sectional view of one embodiment of a vertical column of memory cells.

FIG. 4F is a schematic of a plurality of NAND strings in multiple regions of a same block.

FIG. 5A depicts threshold voltage distributions.

FIG. 5B depicts threshold voltage distributions.

FIG. 5C depicts threshold voltage distributions.

FIG. 5D depicts threshold voltage distributions.

FIG. 6 is a flow chart describing one embodiment of a process for programming non-volatile memory.

FIG. 7A depicts a voltage signal applied to a selected word line during programming.

2

FIG. 7B depicts two program voltage pulses applied to a selected word line during programming and a verify voltage pulse between the two program voltage pulses.

FIG. 7C depicts two program voltage pulses applied to a selected word line during programming and a verify voltage pulse between the two program voltage pulses.

FIG. 8A is a graph describing programming process.

FIG. 8B is a graph describing programming process.

FIG. 9 is a block diagram describing one embodiment of a system for supplying voltages to word lines.

FIG. 10 is a timing diagram depicting voltages applied to NAND strings during a program-verify process.

FIG. 11 includes graphs of word line voltage and system current.

FIG. 12 includes graphs of word line voltage and system current.

FIG. 13 includes graphs of word line voltage and select line voltage.

FIG. 14 is a flow chart describing one embodiment of a process for programming, including performing program-verify.

FIG. 15 is a flow chart describing one embodiment of a process for programming, including performing program-verify.

FIG. 16 is a flow chart describing one embodiment of a process for programming, including performing program-verify.

FIG. 17 is a flow chart describing one embodiment of a process for testing for a higher verify level for a data state.

DETAILED DESCRIPTION

A memory system programs memory cells connected to a selected word line by applying doses of programming and performing program-verify between doses. An efficient and low current program-verify operation includes: while scanning the results of a previous program-verify operation, ramp up voltages on the select lines for the next program-verify operation without waiting for the scan to complete and ramp up voltages on unselected word lines for the next program-verify operation following a step signal (so that voltage applied to the unselected word lines rise in steps) without waiting for the scan to complete.

FIG. 1 is a block diagram of one embodiment of a storage system **100** that implements the proposed technology described herein. In one embodiment, storage system **100** is a solid state drive (“SSD”). Storage system **100** can also be a memory card, USB drive or other type of storage system. The proposed technology is not limited to any one type of memory system. Storage system **100** is connected to host **102**, which can be a computer, server, electronic device (e.g., smart phone, tablet or other mobile device), appliance, or another apparatus that uses memory and has data processing capabilities. In some embodiments, host **102** is separate from, but connected to, storage system **100**. In other embodiments, storage system **100** is embedded within host **102**.

The components of storage system **100** depicted in FIG. 1 are electrical circuits. Storage system **100** includes a memory controller **120** connected to non-volatile memory **130** and local high speed volatile memory **140** (e.g., DRAM). Local high speed volatile memory **140** is used by memory controller **120** to perform certain functions. For example, local high speed volatile memory **140** stores logical to physical address translation tables (“L2P tables”).

Memory controller **120** comprises a host interface **152** that is connected to and in communication with host **102**. In one embodiment, host interface **152** implements a NVMe

Express (NVMe) over PCI Express (PCIe). Other interfaces can also be used, such as SCSI, SATA, etc. Host interface **152** is also connected to a network-on-chip (NOC) **154**. A NOC is a communication subsystem on an integrated circuit. NOC's can span synchronous and asynchronous clock domains or use unlocked asynchronous logic. NOC technology applies networking theory and methods to on-chip communications and brings notable improvements over conventional bus and crossbar interconnections. NOC improves the scalability of systems on a chip (SoC) and the power efficiency of complex SoCs compared to other designs. The wires and the links of the NOC are shared by many signals. A high level of parallelism is achieved because all links in the NOC can operate simultaneously on different data packets. Therefore, as the complexity of integrated subsystems keep growing, a NOC provides enhanced performance (such as throughput) and scalability in comparison with previous communication architectures (e.g., dedicated point-to-point signal wires, shared buses, or segmented buses with bridges). In other embodiments, NOC **154** can be replaced by a bus. Connected to and in communication with NOC **154** is processor **156**, ECC engine **158**, memory interface **160**, and DRAM controller **164**. DRAM controller **164** is used to operate and communicate with local high speed volatile memory **140** (e.g., DRAM). In other embodiments, local high speed volatile memory **140** can be SRAM or another type of volatile memory.

ECC engine **158** performs error correction services. For example, ECC engine **158** performs data encoding and decoding, as per the implemented ECC technique. In one embodiment, ECC engine **158** is an electrical circuit programmed by software. For example, ECC engine **158** can be a processor that can be programmed. In other embodiments, ECC engine **158** is a custom and dedicated hardware circuit without any software. In another embodiment, the function of ECC engine **158** is implemented by processor **156**.

Processor **156** performs the various controller memory operations, such as programming, erasing, reading, and memory management processes. In one embodiment, processor **156** is programmed by firmware. In other embodiments, processor **156** is a custom and dedicated hardware circuit without any software. Processor **156** also implements a translation module, as a software/firmware process or as a dedicated hardware circuit. In many systems, the non-volatile memory is addressed internally to the storage system using physical addresses associated with the one or more memory die. However, the host system will use logical addresses to address the various memory locations. This enables the host to assign data to consecutive logical addresses, while the storage system is free to store the data as it wishes among the locations of the one or more memory die. To implement this system, memory controller **120** (e.g., the translation module) performs address translation between the logical addresses used by the host and the physical addresses used by the memory dies. One example implementation is to maintain tables (i.e., the L2P tables mentioned above) that identify the current translation between logical addresses and physical addresses. An entry in the L2P table may include an identification of a logical address and corresponding physical address. Although logical address to physical address tables (or L2P tables) include the word "tables" they need not literally be tables. Rather, the logical address to physical address tables (or L2P tables) can be any type of data structure. In some examples, the memory space of a storage system is so large that the local memory **140** cannot hold all of the L2P tables. In such a case, the entire set of L2P tables are stored in a memory die

130 and a subset of the L2P tables are cached (L2P cache) in the local high speed volatile memory **140**.

Memory interface **160** communicates with non-volatile memory **130**. In one embodiment, memory interface provides a Toggle Mode interface. Other interfaces can also be used. In some example implementations, memory interface **160** (or another portion of controller **120**) implements a scheduler and buffer for transmitting data to and receiving data from one or more memory die.

In one embodiment, non-volatile memory **130** comprises one or more memory die. FIG. 2A is a functional block diagram of one embodiment of a memory die **200** that comprises non-volatile memory **130**. Each of the one or more memory die of non-volatile memory **130** can be implemented as memory die **200** of FIG. 2A. The components depicted in FIG. 2A are electrical circuits. Memory die **200** includes a memory array **202** that can comprises non-volatile memory cells, as described in more detail below. The array terminal lines of memory array **202** include the various layer(s) of word lines organized as rows, and the various layer(s) of bit lines organized as columns. However, other orientations can also be implemented. Memory die **200** includes row control circuitry **220**, whose outputs **208** are connected to respective word lines of the memory array **202**. Row control circuitry **220** receives a group of M row address signals and one or more various control signals from System Control Logic circuit **260**, and typically may include such circuits as row decoders **222**, array terminal drivers **224**, and block select circuitry **226** for both reading and writing (programming) operations. Row control circuitry **220** may also include read/write circuitry. Memory die **200** also includes column control circuitry **210** including sense amplifier(s) **230** whose input/outputs **206** are connected to respective bit lines of the memory array **202**. Although only single block is shown for array **202**, a memory die can include multiple arrays that can be individually accessed. Column control circuitry **210** receives a group of N column address signals and one or more various control signals from System Control Logic **260**, and typically may include such circuits as column decoders **212**, array terminal receivers or driver circuits **214**, block select circuitry **216**, as well as read/write circuitry, and I/O multiplexers.

System control logic **260** receives data and commands from memory controller **120** and provides output data and status to the host. In some embodiments, the system control logic **260** (which comprises one or more electrical circuits) include state machine **262** that provides die-level control of memory operations. In one embodiment, the state machine **262** is programmable by software. In other embodiments, the state machine **262** does not use software and is completely implemented in hardware (e.g., electrical circuits). In another embodiment, the state machine **262** is replaced by a micro-controller or microprocessor, either on or off the memory chip. System control logic **260** can also include a power control module **264** that controls the power and voltages supplied to the rows and columns of the memory structure **202** during memory operations and may include charge pumps and regulator circuit for creating regulating voltages. System control logic **260** includes storage **366** (e.g., RAM, registers, latches, etc.), which may be used to store parameters for operating the memory array **202**.

Commands and data are transferred between memory controller **120** and memory die **200** via memory controller interface **268** (also referred to as a "communication interface"). Memory controller interface **268** is an electrical interface for communicating with memory controller **120**. Examples of memory controller interface **268** include a

Toggle Mode Interface and an Open NAND Flash Interface (ONFI). Other I/O interfaces can also be used.

In some embodiments, all the elements of memory die **200**, including the system control logic **260**, can be formed as part of a single die. In other embodiments, some or all of the system control logic **260** can be formed on a different die.

In one embodiment, memory structure **202** comprises a three-dimensional memory array of non-volatile memory cells in which multiple memory levels are formed above a single substrate, such as a wafer. The memory structure may comprise any type of non-volatile memory that are monolithically formed in one or more physical levels of memory cells having an active area disposed above a silicon (or other type of) substrate. In one example, the non-volatile memory cells comprise vertical NAND strings with charge-trapping layers.

In another embodiment, memory structure **202** comprises a two-dimensional memory array of non-volatile memory cells. In one example, the non-volatile memory cells are NAND flash memory cells utilizing floating gates. Other types of memory cells (e.g., NOR-type flash memory) can also be used.

The exact type of memory array architecture or memory cell included in memory structure **202** is not limited to the examples above. Many different types of memory array architectures or memory technologies can be used to form memory structure **202**. No particular non-volatile memory technology is required for purposes of the new claimed embodiments proposed herein. Other examples of suitable technologies for memory cells of the memory structure **202** include ReRAM memories (resistive random access memories), magnetoresistive memory (e.g., MRAM, Spin Transfer Torque MRAM, Spin Orbit Torque MRAM), FeRAM, phase change memory (e.g., PCM), and the like. Examples of suitable technologies for memory cell architectures of the memory structure **202** include two dimensional arrays, three dimensional arrays, cross-point arrays, stacked two dimensional arrays, vertical bit line arrays, and the like.

One example of a ReRAM cross-point memory includes reversible resistance-switching elements arranged in cross-point arrays accessed by X lines and Y lines (e.g., word lines and bit lines). In another embodiment, the memory cells may include conductive bridge memory elements. A conductive bridge memory element may also be referred to as a programmable metallization cell. A conductive bridge memory element may be used as a state change element based on the physical relocation of ions within a solid electrolyte. In some cases, a conductive bridge memory element may include two solid metal electrodes, one relatively inert (e.g., tungsten) and the other electrochemically active (e.g., silver or copper), with a thin film of the solid electrolyte between the two electrodes. As temperature increases, the mobility of the ions also increases causing the programming threshold for the conductive bridge memory cell to decrease. Thus, the conductive bridge memory element may have a wide range of programming thresholds over temperature.

Another example is magnetoresistive random access memory (MRAM) that stores data by magnetic storage elements. The elements are formed from two ferromagnetic layers, each of which can hold a magnetization, separated by a thin insulating layer. One of the two layers is a permanent magnet set to a particular polarity; the other layer's magnetization can be changed to match that of an external field to store memory. A memory device is built from a grid of such memory cells. In one embodiment for programming, each memory cell lies between a pair of write lines arranged

at right angles to each other, parallel to the cell, one above and one below the cell. When current is passed through them, an induced magnetic field is created. MRAM based memory embodiments will be discussed in more detail below.

Phase change memory (PCM) exploits the unique behavior of chalcogenide glass. One embodiment uses a GeTe—Sb₂Te₃ super lattice to achieve non-thermal phase changes by simply changing the co-ordination state of the Germanium atoms with a laser pulse (or light pulse from another source). Therefore, the doses of programming are laser pulses. The memory cells can be inhibited by blocking the memory cells from receiving the light. In other PCM embodiments, the memory cells are programmed by current pulses. Note that the use of “pulse” in this document does not require a square pulse but includes a (continuous or non-continuous) vibration or burst of sound, current, voltage light, or another wave. These memory elements within the individual selectable memory cells, or bits, may include a further series element that is a selector, such as an ovonic threshold switch or metal insulator substrate.

A person of ordinary skill in the art will recognize that the technology described herein is not limited to a single specific memory structure, memory construction or material composition, but covers many relevant memory structures within the spirit and scope of the technology as described herein and as understood by one of ordinary skill in the art.

The elements of FIG. **2A** can be grouped into two parts: (1) memory structure **202** and (2) peripheral circuitry, which includes all of the other components depicted in FIG. **2A**. An important characteristic of a memory circuit is its capacity, which can be increased by increasing the area of the memory die of storage system **100** that is given over to the memory structure **202**; however, this reduces the area of the memory die available for the peripheral circuitry. This can place quite severe restrictions on these elements of the peripheral circuitry. For example, the need to fit sense amplifier circuits within the available area can be a significant restriction on sense amplifier design architectures. With respect to the system control logic **260**, reduced availability of area can limit the available functionalities that can be implemented on-chip. Consequently, a basic trade-off in the design of a memory die for the storage system **100** is the amount of area to devote to the memory structure **202** and the amount of area to devote to the peripheral circuitry.

Another area in which the memory structure **202** and the peripheral circuitry are often at odds is in the processing involved in forming these regions, since these regions often involve differing processing technologies and the trade-off in having differing technologies on a single die. For example, when the memory structure **202** is NAND flash, this is an NMOS structure, while the peripheral circuitry is often CMOS based. For example, elements such as sense amplifier circuits, charge pumps, logic elements in a state machine, and other peripheral circuitry in system control logic **260** often employ PMOS devices. Processing operations for manufacturing a CMOS die will differ in many aspects from the processing operations optimized for an NMOS flash NAND memory or other memory cell technologies.

To improve upon these limitations, embodiments described below can separate the elements of FIG. **2A** onto separately formed dies that are then bonded together. More specifically, the memory structure **202** can be formed on one die (referred to as the memory die) and some or all of the peripheral circuitry elements, including one or more control circuits, can be formed on a separate die (referred to as the

control die). For example, a memory die can be formed of just the memory elements, such as the array of memory cells of flash NAND memory, MRAM memory, PCM memory, ReRAM memory, or other memory type. Some or all of the peripheral circuitry, even including elements such as decoders and sense amplifiers, can then be moved on to a separate control die. This allows each of the memory die to be optimized individually according to its technology. For example, a NAND memory die can be optimized for an NMOS based memory array structure, without worrying about the CMOS elements that have now been moved onto a control die that can be optimized for CMOS processing. This allows more space for the peripheral elements, which can now incorporate additional capabilities that could not be readily incorporated were they restricted to the margins of the same die holding the memory cell array. The two die can then be bonded together in a bonded multi-die memory circuit, with the array on the one die connected to the periphery elements on the other die. Although the following will focus on a bonded memory circuit of one memory die and one control die, other embodiments can use more die, such as two memory die and one control die, for example.

FIG. 2B shows an alternative arrangement to that of FIG. 2A which may be implemented using wafer-to-wafer bonding to provide a bonded die pair. FIG. 2B depicts a functional block diagram of one embodiment of an integrated memory assembly 207. One or more integrated memory assemblies 207 may be used to implement the non-volatile memory 130 of storage system 100. The integrated memory assembly 207 includes two types of semiconductor die (or more succinctly, “die”). Memory die 201 includes memory structure 202. Memory structure 202 includes non-volatile memory cells. Control die 211 includes control circuitry 260, 210, and 220 (as described above). In some embodiments, control die 211 is configured to connect to the memory structure 202 in the memory die 201. In some embodiments, the memory die 201 and the control die 211 are bonded together.

FIG. 2B shows an example of the peripheral circuitry, including control circuits, formed in a peripheral circuit or control die 211 coupled to memory structure 202 formed in memory die 201. Common components are labelled similarly to FIG. 2A. System control logic 260, row control circuitry 220, and column control circuitry 210 are located in control die 211. In some embodiments, all or a portion of the column control circuitry 210 and all or a portion of the row control circuitry 220 are located on the memory die 201. In some embodiments, some of the circuitry in the system control logic 260 is located on the on the memory die 201.

System control logic 260, row control circuitry 220, and column control circuitry 210 may be formed by a common process (e.g., CMOS process), so that adding elements and functionalities, such as ECC, more typically found on a memory controller 120 may require few or no additional process steps (i.e., the same process steps used to fabricate controller 120 may also be used to fabricate system control logic 260, row control circuitry 220, and column control circuitry 210). Thus, while moving such circuits from a die such as memory 2 die 201 may reduce the number of steps needed to fabricate such a die, adding such circuits to a die such as control die 211 may not require many additional process steps. The control die 211 could also be referred to as a CMOS die, due to the use of CMOS technology to implement some or all of control circuitry 260, 210, 220.

FIG. 2B shows column control circuitry 210 including sense amplifier(s) 230 on the control die 211 coupled to memory structure 202 on the memory die 201 through

electrical paths 206. For example, electrical paths 206 may provide electrical connection between column decoder 212, driver circuitry 214, and block select 216 and bit lines of memory structure 202. Electrical paths may extend from column control circuitry 210 in control die 211 through pads on control die 211 that are bonded to corresponding pads of the memory die 201, which are connected to bit lines of memory structure 202. Each bit line of memory structure 202 may have a corresponding electrical path in electrical paths 206, including a pair of bond pads, which connects to column control circuitry 210. Similarly, row control circuitry 220, including row decoder 222, array drivers 224, and block select 226 are coupled to memory structure 202 through electrical paths 208. Each of electrical path 208 may correspond to a word line, dummy word line, or select gate line. Additional electrical paths may also be provided between control die 211 and memory die 201.

For purposes of this document, the phrases “a control circuit” or “one or more control circuits” can include any one of or any combination of memory controller 120, state machine 262, all or a portion of system control logic 260, all or a portion of row control circuitry 220, all or a portion of column control circuitry 210, a microcontroller, a microprocessor, and/or other similar functioned circuits. The control circuit can include hardware only or a combination of hardware and software (including firmware). For example, a controller programmed by firmware to perform the functions described herein is one example of a control circuit. A control circuit can include a processor, FGA, ASIC, integrated circuit, or other type of circuit.

In some embodiments, there is more than one control die 211 and more than one memory die 201 in an integrated memory assembly 207. In some embodiments, the integrated memory assembly 207 includes a stack of multiple control die 211 and multiple memory die 201. FIG. 3A depicts a side view of an embodiment of an integrated memory assembly 207 stacked on a substrate 271 (e.g., a stack comprising control dies 211 and memory dies 201). The integrated memory assembly 207 has three control dies 211 and three memory dies 201. In some embodiments, there are more than three memory dies 201 and more than three control die 211.

Each control die 211 is affixed (e.g., bonded) to at least one of the memory dies 201. Some of the bond pads 282/284 are depicted. There may be many more bond pads. A space between two dies 201, 211 that are bonded together is filled with a solid layer 280, which may be formed from epoxy or other resin or polymer. This solid layer 280 protects the electrical connections between the dies 201, 211, and further secures the dies together. Various materials may be used as solid layer 280, but in embodiments, it may be Hysol epoxy resin from Henkel Corp., having offices in California, USA.

The integrated memory assembly 207 may for example be stacked with a stepped offset, leaving the bond pads at each level uncovered and accessible from above. Wire bonds 270 connected to the bond pads connect the control die 211 to the substrate 271. A number of such wire bonds may be formed across the width of each control die 211 (i.e., into the page of FIG. 3A).

A memory die through silicon via (TSV) 276 may be used to route signals through a memory die 201. A control die through silicon via (TSV) 278 may be used to route signals through a control die 211. The TSVs 276, 278 may be formed before, during or after formation of the integrated circuits in the semiconductor dies 201, 211. The TSVs may be formed by etching holes through the wafers. The holes may then be lined with a barrier against metal diffusion. The

barrier layer may in turn be lined with a seed layer, and the seed layer may be plated with an electrical conductor such as copper, although other suitable materials such as aluminum, tin, nickel, gold, doped polysilicon, and alloys or combinations thereof may be used.

Solder balls **272** may optionally be affixed to contact pads **274** on a lower surface of substrate **271**. The solder balls **272** may be used to couple the integrated memory assembly **207** electrically and mechanically to a host device such as a printed circuit board. Solder balls **272** may be omitted where the integrated memory assembly **207** is to be used as an LGA package. The solder balls **272** may form a part of the interface between integrated memory assembly **207** and memory controller **120**.

FIG. 3B depicts a side view of another embodiment of an integrated memory assembly **207** stacked on a substrate **271**. The integrated memory assembly **207** of FIG. 3B has three control die **211** and three memory die **201**. In some embodiments, there are many more than three memory dies **201** and many more than three control dies **211**. In this example, each control die **211** is bonded to at least one memory die **201**. Optionally, a control die **211** may be bonded to two or more memory die **201**.

Some of the bond pads **282**, **284** are depicted. There may be many more bond pads. A space between two dies **201**, **211** that are bonded together is filled with a solid layer **280**, which may be formed from epoxy or other resin or polymer. In contrast to the example in FIG. 3A, the integrated memory assembly **207** in FIG. 3B does not have a stepped offset. A memory die through silicon via (TSV) **276** may be used to route signals through a memory die **201**. A control die through silicon via (TSV) **278** may be used to route signals through a control die **211**.

Solder balls **272** may optionally be affixed to contact pads **274** on a lower surface of substrate **271**. The solder balls **272** may be used to couple the integrated memory assembly **207** electrically and mechanically to a host device such as a printed circuit board. Solder balls **272** may be omitted where the integrated memory assembly **207** is to be used as an LGA package.

As has been briefly discussed above, the control die **211** and the memory die **201** may be bonded together. Bond pads on each die **201**, **211** may be used to bond the two dies together. In some embodiments, the bond pads are bonded directly to each other, without solder or other added material, in a so-called Cu-to-Cu bonding process. In a Cu-to-Cu bonding process, the bond pads are controlled to be highly planar and formed in a highly controlled environment largely devoid of ambient particulates that might otherwise settle on a bond pad and prevent a close bond. Under such properly controlled conditions, the bond pads are aligned and pressed against each other to form a mutual bond based on surface tension. Such bonds may be formed at room temperature, though heat may also be applied. In embodiments using Cu-to-Cu bonding, the bond pads may be about 5 μm square and spaced from each other with a pitch of 5 μm to 5 μm . While this process is referred to herein as Cu-to-Cu bonding, this term may also apply even where the bond pads are formed of materials other than Cu.

When the area of bond pads is small, it may be difficult to bond the semiconductor dies together. The size of, and pitch between, bond pads may be further reduced by providing a film layer on the surfaces of the semiconductor dies including the bond pads. The film layer is provided around the bond pads. When the dies are brought together, the bond pads may bond to each other, and the film layers on the respective dies may bond to each other. Such a bonding

technique may be referred to as hybrid bonding. In embodiments using hybrid bonding, the bond pads may be about 5 μm square and spaced from each other with a pitch of 1 μm to 5 μm . Bonding techniques may be used providing bond pads with even smaller (or greater) sizes and pitches.

Some embodiments may include a film on surface of the dies **201**, **211**. Where no such film is initially provided, a space between the dies may be under filled with an epoxy or other resin or polymer. The under-fill material may be applied as a liquid which then hardens into a solid layer. This under-fill step protects the electrical connections between the dies **201**, **211**, and further secures the dies together. Various materials may be used as under-fill material, but in embodiments, it may be Hysol epoxy resin from Henkel Corp., having offices in California, USA.

FIG. 4 is a perspective view of a portion of one example embodiment of a monolithic three dimensional memory array/structure that can comprise memory structure **202**, which includes a plurality non-volatile memory cells arranged as vertical NAND strings. For example, FIG. 4 shows a portion **400** of one block of memory. The structure depicted includes a set of bit lines BL positioned above a stack **401** of alternating dielectric layers and conductive layers. For example purposes, one of the dielectric layers is marked as D and one of the conductive layers (also called word line layers) is marked as W. The number of alternating dielectric layers and conductive layers can vary based on specific implementation requirements. As will be explained below, in one embodiment the alternating dielectric layers and conductive layers are divided into four or five (or a different number of) regions by isolation regions IR. FIG. 4 shows one isolation region IR separating two regions. Below the alternating dielectric layers and word line layers is a source line layer SL. Memory holes are formed in the stack of alternating dielectric layers and conductive layers. For example, one of the memory holes is marked as MH. Note that in FIG. 4, the dielectric layers are depicted as see-through so that the reader can see the memory holes positioned in the stack of alternating dielectric layers and conductive layers. In one embodiment, NAND strings are formed by filling the memory hole with materials including a charge-trapping material to create a vertical column of memory cells. Each memory cell can store one or more bits of data. Thus, the non-volatile memory cells are arranged in memory holes. More details of the three dimensional monolithic memory array that comprises memory structure **202** is provided below.

FIG. 4A is a block diagram explaining one example organization of memory structure **202**, which is divided into two planes **402** and **404**. Each plane is then divided into M blocks. In one example, each plane has about 2000 blocks. However, different numbers of blocks and planes can also be used. In one embodiment, a block of memory cells is a unit of erase. That is, all memory cells of a block are erased together. In other embodiments, blocks can be divided into sub-blocks and the sub-blocks can be the unit of erase. Memory cells can also be grouped into blocks for other reasons, such as to organize the memory structure to enable the signaling and selection circuits. In some embodiments, a block represents a groups of connected memory cells as the memory cells of a block share a common set of word lines. For example, the word lines for a block are all connected to all of the vertical NAND strings for that block. Although FIG. 4A shows two planes **402/404**, more or less than two planes can be implemented. In some embodiments, memory structure **202** includes eight planes.

11

FIGS. 4B-4G depict an example three dimensional (“3D”) NAND structure that corresponds to the structure of FIG. 4 and can be used to implement memory structure 202 of FIGS. 2A and 2B. FIG. 4B is a block diagram depicting a top view of a portion 406 of Block 2 of plane 402. As can be seen from FIG. 4B, the block depicted in FIG. 4B extends in the direction of 432. In one embodiment, the memory array has many layers; however, FIG. 4B only shows the top layer.

FIG. 4B depicts a plurality of circles that represent the memory holes, which are also referred to as vertical columns. Each of the memory holes/vertical columns include multiple select transistors (also referred to as a select gate or selection gate) and multiple memory cells. In one embodiment, each memory hole/vertical column implements a NAND string. For example, FIG. 4B labels a subset of the memory holes/vertical columns/NAND strings 432, 436, 446, 456, 462, 466, 472, 474 and 476.

FIG. 4B also depicts a set of bit lines 415, including bit lines 411, 412, 413, 414, . . . 419. FIG. 4B shows twenty four bit lines because only a portion of the block is depicted. It is contemplated that more than twenty four bit lines connected to memory holes/vertical columns of the block. Each of the circles representing memory holes/vertical columns has an “x” to indicate its connection to one bit line. For example, bit line 411 is connected to memory holes/vertical columns 436, 446, 456, 466 and 476.

The block depicted in FIG. 4B includes a set of isolation regions 482, 484, 486 and 488, which are formed of SiO₂; however, other dielectric materials can also be used. Isolation regions 482, 484, 486 and 488 serve to divide the top layers of the block into five regions; for example, the top layer depicted in FIG. 4B is divided into regions 430, 440, 450, 460 and 470. In one embodiment, the isolation regions only divide the layers used to implement select gates so that NAND strings in different regions can be independently selected. In one example implementation, a bit line connects to one memory hole/vertical column/NAND string in each of regions 430, 440, 450, 460 and 470. In that implementation, each block has twenty four rows of active columns and each bit line connects to five rows in each block. In one embodiment, all of the five memory holes/vertical columns/NAND strings connected to a common bit line are connected to the same set of word lines; therefore, the system uses the drain side selection lines to choose one (or another subset) of the five to be subjected to a memory operation (program, verify, read, and/or erase).

FIG. 4B also shows Line Interconnects LI, which are metal connections to the source line SL from above the memory array. Line Interconnects LI are positioned adjacent regions 430 and 470.

Although FIG. 4B shows each region 430, 440, 450, 460 and 470 having four rows of memory holes/vertical columns, five regions and twenty four rows of memory holes/vertical columns in a block, those exact numbers are an example implementation. Other embodiments may include more or less regions per block, more or less rows of memory holes/vertical columns per region and more or less rows of vertical columns per block. FIG. 4B also shows the memory holes/vertical columns being staggered. In other embodiments, different patterns of staggering can be used. In some embodiments, the memory holes/vertical columns are not staggered.

FIG. 4C depicts a portion of one embodiment of a three dimensional memory structure 202 showing a cross-sectional view along line AA of FIG. 4B. This cross sectional view cuts through memory holes/vertical columns (NAND

12

strings) 472 and 474 of region 470 (see FIG. 4B). The structure of FIG. 4C includes three drain side select layers SGD0, SGD1 and SGD2; three source side select layers SGS0, SGS1, and SGS2; three drain side GIDL generation transistor layers SGDT0, SGDT1, and SGDT2; three source side GIDL generation transistor layers SGSB0, SGSB1, and SGSB2; four drain side dummy word line layers DD0, DD1, DD2 and DD3; four source side dummy word line layers DS0, DS1, DS2 and DS3; dummy word line layers DU and DL; one hundred and sixty two word line layers WL0-WL161 for connecting to data memory cells, and dielectric layers DL. Other embodiments can implement more or less than the numbers described above for FIG. 4C. In one embodiment, SGD0, SGD1 and SGD2 are connected together; and SGS0, SGS1 and SGS2 are connected together. In other embodiments, more or less number of SGDs (greater or lesser than three) are connected together, and more or less number of SGSs (greater or lesser than three) connected together.

In one embodiment, erasing the memory cells is performed using gate induced drain leakage (GIDL), which includes generating charge carriers at the GIDL generation transistors such that the carriers get injected into the charge trapping layers of the NAND strings to change threshold voltage of the memory cells. FIG. 4C shows three GIDL generation transistors at each end of the NAND string; however, in other embodiments there are more or less than three. Embodiments that use GIDL at both sides of the NAND string may have GIDL generation transistors at both sides. Embodiments that use GIDL at only the drain side of the NAND string may have GIDL generation transistors only at the drain side. Embodiments that use GIDL at only the source side of the NAND string may have GIDL generation transistors only at the source side.

FIG. 4C shows three GIDL generation transistors at each end of the NAND string. It is likely that charge carriers are only generated by GIDL at one of the three GIDL generation transistors at each end of the NAND string. Based on process variances during manufacturing, it is likely that one of the three GIDL generation transistors at an end of the NAND string is best suited for GIDL. For example, the GIDL generation transistors have an abrupt pn junction to generate the charge carriers for GIDL and, during fabrication, a phosphorous diffusion is performed at the polysilicon channel of the GIDL generation transistors. In some cases, the GIDL generation transistor with the shallowest phosphorous diffusion is the GIDL generation transistor that generates the charge carriers during erase. However, in some embodiments charge carriers can be generated by GIDL at multiple GIDL generation transistors at a particular side of the NAND string.

Memory holes/Vertical columns 472 and 474 are depicted protruding through the drain side select layers, source side select layers, dummy word line layers, GIDL generation transistor layers and word line layers. In one embodiment, each memory hole/vertical column comprises a vertical NAND string. Below the memory holes/vertical columns and the layers listed below is substrate 453, an insulating film 454 on the substrate, and source line SL. The NAND string of memory hole/vertical column 472 has a source end at a bottom of the stack and a drain end at a top of the stack. As in agreement with FIG. 4B, FIG. 4C show vertical memory hole/column 472 connected to bit line 414 via connector 417.

For ease of reference, drain side select layers; source side select layers, dummy word line layers, GIDL generation transistor layers and data word line layers collectively are

referred to as the conductive layers. In one embodiment, the conductive layers are made from a combination of TiN and Tungsten. In other embodiments, other materials can be used to form the conductive layers, such as doped polysilicon, metal such as Tungsten, metal silicide, such as nickel silicide, tungsten silicide, aluminum silicide or the combination thereof. In some embodiments, different conductive layers can be formed from different materials. Between conductive layers are dielectric layers DL. In one embodiment, the dielectric layers are made from SiO₂. In other embodiments, other dielectric materials can be used to form the dielectric layers.

The non-volatile memory cells are formed along memory holes/vertical columns which extend through alternating conductive and dielectric layers in the stack. In one embodiment, the memory cells are arranged in NAND strings. The word line layers WL0-WL161 connect to memory cells (also called data memory cells). Dummy word line layers connect to dummy memory cells. A dummy memory cell does not store and is not eligible to store host data (data provided from the host, such as data from a user of the host), while a data memory cell is eligible to store host data. In some embodiments, data memory cells and dummy memory cells may have a same structure. Drain side select layers SGD0, SGD1, and SGD2 are used to electrically connect and disconnect NAND strings from bit lines. Source side select layers SGS0, SGS1, and SGS2 are used to electrically connect and disconnect NAND strings from the source line SL.

FIG. 4C shows that the memory array is implemented as a two tier architecture, with the tiers separated by a Joint area. In one embodiment it is expensive and/or challenging to etch so many word line layers intermixed with dielectric layers. To ease this burden, one embodiment includes laying down a first stack of word line layers (e.g., WL0-WL80) alternating with dielectric layers, laying down the Joint area, and laying down a second stack of word line layers (e.g., WL81-WL161) alternating with dielectric layers. The Joint areas are positioned between the first stack and the second stack. In one embodiment, the Joint areas are made from the same materials as the word line layers. In other embodiments, there can be no Joint area or multiple Joint areas.

FIG. 4D depicts a portion of one embodiment of a three dimensional memory structure 202 showing a cross-sectional view along line BB of FIG. 4B. This cross sectional view cuts through memory holes/vertical columns (NAND strings) 432 and 434 of region 430 (see FIG. 4B). FIG. 4D shows the same alternating conductive and dielectric layers as FIG. 4C. FIG. 4D also shows isolation region 482. Isolation regions 482, 484, 486 and 488 occupy space that would have been used for a portion of the memory holes/vertical columns/NAND strings. For example, isolation region 482 occupies space that would have been used for a portion of memory hole/vertical column 434. More specifically, a portion (e.g., half the diameter) of vertical column 434 has been removed in layers SGDT0, SGDT1, SGDT2, SGD0, SGD1, SGD2, DD0, DD1 and DD2 to accommodate isolation region 482. Thus, while most of the vertical column 434 is cylindrical (with a circular cross section), the portion of vertical column 434 in layers SGDT0, SGDT1, SGDT2, SGD0, SGD1, SGD2, DD0, DD1 and DD2 has a semi-circular cross section. In one embodiment, after the stack of alternating conductive and dielectric layers is formed, the stack is etched to create space for the isolation region and that space is then filled in with SiO₂.

FIG. 4E depicts a cross sectional view of region 429 of FIG. 4C that includes a portion of memory hole/vertical

column 472. In one embodiment, the memory holes/vertical columns are round; however, in other embodiments other shapes can be used. In one embodiment, memory hole/vertical column 472 includes an inner core layer 490 that is made of a dielectric, such as SiO₂. Other materials can also be used. Surrounding inner core 490 is polysilicon channel 491. Materials other than polysilicon can also be used. Note that it is the channel 491 that connects to the bit line and the source line. Surrounding channel 491 is a tunneling dielectric 492. In one embodiment, tunneling dielectric 492 has an ONO structure. Surrounding tunneling dielectric 492 is charge trapping layer 493, such as (for example) Silicon Nitride. Other memory materials and structures can also be used. The technology described herein is not limited to any particular material or structure.

FIG. 4E depicts dielectric layers DL as well as word line layers WL160, WL159, WL158, WL157, and WL156. Each of the word line layers includes a word line region 496 surrounded by an aluminum oxide layer 497, which is surrounded by a blocking oxide layer 498. In other embodiments, the blocking oxide layer can be a vertical layer parallel and adjacent to charge trapping layer 493. The physical interaction of the word line layers with the vertical column forms the memory cells. Thus, a memory cell, in one embodiment, comprises channel 491, tunneling dielectric 492, charge trapping layer 493, blocking oxide layer 498, aluminum oxide layer 497 and word line region 496. For example, word line layer WL160 and a portion of memory hole/vertical column 472 comprise a memory cell MC1. Word line layer WL159 and a portion of memory hole/vertical column 472 comprise a memory cell MC2. Word line layer WL158 and a portion of memory hole/vertical column 472 comprise a memory cell MC3. Word line layer WL157 and a portion of memory hole/vertical column 472 comprise a memory cell MC4. Word line layer WL156 and a portion of memory hole/vertical column 472 comprise a memory cell MC5. In other architectures, a memory cell may have a different structure; however, the memory cell would still be the storage unit.

When a memory cell is programmed, electrons are stored in a portion of the charge trapping layer 493 which is associated with (e.g. in) the memory cell. These electrons are drawn into the charge trapping layer 493 from the channel 491, through the tunneling dielectric 492, in response to an appropriate voltage on word line region 496. The threshold voltage (V_{th}) of a memory cell is increased in proportion to the amount of stored charge. In one embodiment, the programming is achieved through Fowler-Nordheim tunneling of the electrons into the charge trapping layer. During an erase operation, the electrons return to the channel or holes are injected into the charge trapping layer to recombine with electrons. In one embodiment, erasing is achieved using hole injection into the charge trapping layer via a physical mechanism such as GIDL.

FIG. 4F is a schematic diagram of a portion of the memory array 202 depicted in in FIGS. 4-4E. FIG. 4F shows physical data word lines WL0-WL161 running across the entire block. The structure of FIG. 4F corresponds to a portion 406 in Block 2 of FIG. 4A, including bit line 411. Within the block, in one embodiment, each bit line is connected to five NAND strings. Thus, FIG. 4F shows bit line connected to NAND string NS0 (which corresponds to memory hole/vertical column 436), NAND string NS1 (which corresponds to memory hole/vertical column 446), NAND string NS2 (which corresponds to vertical column 456), NAND string NS3 (which corresponds to memory hole/vertical column 466), and NAND string NS4 (which

15

corresponds to memory hole/vertical column 476). As mentioned above, in one embodiment, SGD0, SGD1 and SGD2 are connected together to operate as a single logical select gate for each region separated by isolation regions (482, 484, 486 and 488) to form SGD-s0, SGD-s1, SGD-s2, SGD-s3, and SGD-s4. SGS0, SGS1 and SGS2 are also connected together to operate as a single logical select gate that is represented in FIG. 4F as SGS. Although the select gates SGD-s0, SGD-s1, SGD-s2, SGD-s3, and SGD-s4 are isolated from each other due to the isolation regions, the data word lines WL0-WL161 of each region are connected together. Thus, data word lines WL0-WL161 are connected to NAND strings (and memory cells) of each (or every) region (430, 440, 450, 460, 470) of a block.

The isolation regions (482, 484, 486 and 488) are used to allow for separate control of regions 430, 440, 450, 460, 470. A first region corresponds to those vertical NAND strings controlled by SGD-s0. A second region corresponds to those vertical NAND strings controlled by SGD-s1. A third region corresponds to those vertical NAND strings controlled by SGD-s2. A fourth region corresponds to those vertical NAND strings controlled by SGD-s3. A fifth region corresponds to those vertical NAND strings controlled by SGD-s4.

FIG. 4F only shows NAND strings connected to bit line 411. However, a full schematic of the block would show every bit line and five vertical NAND strings (that are in separate regions) connected to each bit line.

Although the example memories of FIGS. 4-4F are three dimensional memory structure that includes vertical NAND strings with charge-trapping material, other (2D and 3D) memory structures can also be used with the technology described herein.

The memory systems discussed above can be erased, programmed and read. At the end of a successful programming process, the threshold voltages of the memory cells should be within one or more distributions of threshold voltages for programmed memory cells or within a distribution of threshold voltages for erased memory cells, as appropriate. FIG. 5A is a graph of threshold voltage versus number of memory cells, and illustrates example threshold voltage distributions for the memory array when each memory cell stores one bit of data per memory cell. Memory cells that store one bit of data per memory cell data are referred to as single level cells ("SLC"). The data stored in SLC memory cells is referred to as SLC data; therefore, SLC data comprises one bit per memory cell. Data stored as one bit per memory cell is SLC data. FIG. 5A shows two threshold voltage distributions: E and P. Threshold voltage distribution E corresponds to an erased data state. Threshold voltage distribution P corresponds to a programmed data state. Memory cells that have threshold voltages in threshold voltage distribution E are, therefore, in the erased data state (e.g., they are erased). Memory cells that have threshold voltages in threshold voltage distribution P are, therefore, in the programmed data state (e.g., they are programmed). In one embodiment, erased memory cells store data "1" and programmed memory cells store data "0." FIG. 5A depicts read reference voltage V_r . By testing (e.g., performing one or more sense operations) whether the threshold voltage of a given memory cell is above or below V_r , the system can determine a memory cells is erased (state E) or programmed (state P). FIG. 5A also depicts verify reference voltage V_v . In some embodiments, when programming memory cells to data state P, the system will test whether those memory cells have a threshold voltage greater than or equal to V_v .

16

FIGS. 5B-D illustrate example threshold voltage distributions for the memory array when each memory cell stores multiple bit per memory cell data. Memory cells that store multiple bit per memory cell data are referred to as multi-level cells ("MLC"). The data stored in MLC memory cells is referred to as MLC data; therefore, MLC data comprises multiple bits per memory cell. Data stored as multiple bits of data per memory cell is MLC data. In the example embodiment of FIG. 5B, each memory cell stores two bits of data. Other embodiments may use other data capacities per memory cell (e.g., such as three, four, or five bits of data per memory cell).

FIG. 5B shows a first threshold voltage distribution E for erased memory cells. Three threshold voltage distributions A, B and C for programmed memory cells are also depicted. In one embodiment, the threshold voltages in the distribution E are negative and the threshold voltages in distributions A, B and C are positive. Each distinct threshold voltage distribution of FIG. 5B corresponds to predetermined values for the set of data bits. In one embodiment, each bit of data of the two bits of data stored in a memory cell are in different logical pages, referred to as a lower page (LP) and an upper page (UP). In other embodiments, all bits of data stored in a memory cell are in a common logical page. The specific relationship between the data programmed into the memory cell and the threshold voltage levels of the cell depends upon the data encoding scheme adopted for the cells. Table 1 provides an example encoding scheme.

TABLE 1

	E	A	B	C
LP	1	0	0	1
UP	1	1	0	0

In one embodiment, known as full sequence programming, memory cells can be programmed from the erased data state E directly to any of the programmed data states A, B or C using the process of FIG. 6 (discussed below). For example, a population of memory cells to be programmed may first be erased so that all memory cells in the population are in erased data state E. Then, a programming process is used to program memory cells directly into data states A, B, and/or C. For example, while some memory cells are being programmed from data state E to data state A, other memory cells are being programmed from data state E to data state B and/or from data state E to data state C. The arrows of FIG. 5B represent the full sequence programming. In some embodiments, data states A-C can overlap, with memory controller 120 (or control die 211) relying on error correction to identify the correct data being stored.

FIG. 5C depicts example threshold voltage distributions for memory cells where each memory cell stores three bits of data per memory cells (which is another example of MLC data). FIG. 5C shows eight threshold voltage distributions, corresponding to eight data states. The first threshold voltage distribution (data state) Er represents memory cells that are erased. The other seven threshold voltage distributions (data states) A-G represent memory cells that are programmed and, therefore, are also called programmed states. Each threshold voltage distribution (data state) corresponds to predetermined values for the set of data bits. The specific relationship between the data programmed into the memory cell and the threshold voltage levels of the cell depends upon the data encoding scheme adopted for the cells. In one embodiment, data values are assigned to the threshold

voltage ranges using a Gray code assignment so that if the threshold voltage of a memory erroneously shifts to its neighboring physical state, only one bit will be affected. Table 2 provides an example of an encoding scheme for embodiments in which each bit of data of the three bits of data stored in a memory cell are in different logical pages, referred to as a lower page (LP), middle page (MP) and an upper page (UP).

TABLE 2

	Er	A	B	C	D	E	F	G
UP	1	1	1	0	0	0	0	1
MP	1	1	0	0	1	1	0	0
LP	1	0	0	0	0	1	1	1

FIG. 5C shows seven read reference voltages, VrA, VrB, VrC, VrD, VrE, VrF, and VrG for reading data from memory cells. By testing (e.g., performing sense operations) whether the threshold voltage of a given memory cell is above or below the seven read reference voltages, the system can determine what data state (i.e., A, B, C, D, . . .) a memory cell is in.

FIG. 5C also shows seven verify reference voltages, VvA, VvB, VvC, VvD, VvE, VvF, and VvG. In some embodiments, when programming memory cells to data state A, the system will test whether those memory cells have a threshold voltage greater than or equal to VvA. When programming memory cells to data state B, the system will test whether the memory cells have threshold voltages greater than or equal to VvB. When programming memory cells to data state C, the system will determine whether memory cells have their threshold voltage greater than or equal to VvC. When programming memory cells to data state D, the system will test whether those memory cells have a threshold voltage greater than or equal to VvD. When programming memory cells to data state E, the system will test whether those memory cells have a threshold voltage greater than or equal to VvE. When programming memory cells to data state F, the system will test whether those memory cells have a threshold voltage greater than or equal to VvF. When programming memory cells to data state G, the system will test whether those memory cells have a threshold voltage greater than or equal to VvG. FIG. 5C also shows Vev, which is an erase verify reference voltage to test whether a memory cell has been properly erased.

In an embodiment that utilizes full sequence programming, memory cells can be programmed from the erased data state Er directly to any of the programmed data states A-G using the process of FIG. 6 (discussed below). For example, a population of memory cells to be programmed may first be erased so that all memory cells in the population are in erased data state Er. Then, a programming process is used to program memory cells directly into data states A, B, C, D, E, F, and/or G. For example, while some memory cells are being programmed from data state Er to data state A, other memory cells are being programmed from data state Er to data state B and/or from data state Er to data state C, and so on. The arrows of FIG. 5C represent the full sequence programming. In some embodiments, data states A-G can overlap, with control die 211 and/or memory controller 120 relying on error correction to identify the correct data being stored. Note that in some embodiments, rather than using full sequence programming, the system can use multi-pass programming processes known in the art.

In general, during verify operations and read operations, the selected word line is connected to a voltage (one example of a reference signal), a level of which is specified for each read operation (e.g., see read compare voltages/levels VrA, VrB, VrC, VrD, VrE, VrF, and VrG, of FIG. 5C) or verify operation (e.g. see verify target voltages/levels VvA, VvB, VvC, VvD, VvE, VvF, and VvG of FIG. 5C) in order to determine whether a threshold voltage of the concerned memory cell has reached such level. After applying the word line voltage, the conduction current of the memory cell is measured to determine whether the memory cell turned on (conducted current) in response to the voltage applied to the word line. If the conduction current is measured to be greater than a certain value, then it is assumed that the memory cell turned on and the voltage applied to the word line is greater than the threshold voltage of the memory cell. If the conduction current is not measured to be greater than the certain value, then it is assumed that the memory cell did not turn on and the voltage applied to the word line is not greater than the threshold voltage of the memory cell. During a read or verify process, the unselected memory cells are provided with one or more read pass voltages (also referred to as bypass voltages) at their control gates so that these memory cells will operate as pass gates (e.g., conducting current regardless of whether they are programmed or erased).

There are many ways to measure the conduction current of a memory cell during a read or verify operation. In one example, the conduction current of a memory cell is measured by the rate it discharges or charges a dedicated capacitor in the sense amplifier. In another example, the conduction current of the selected memory cell allows (or fails to allow) the NAND string that includes the memory cell to discharge a corresponding bit line. The voltage on the bit line is measured after a period of time to see whether it has been discharged or not. Note that the technology described herein can be used with different methods known in the art for verifying/reading. Other read and verify techniques known in the art can also be used.

FIG. 5D depicts threshold voltage distributions when each memory cell stores four bits of data, which is another example of MLC data. FIG. 5D depicts that there may be some overlap between the threshold voltage distributions (data states) S0-S15. The overlap may occur due to factors such as memory cells losing charge (and hence dropping in threshold voltage). Program disturb can unintentionally increase the threshold voltage of a memory cell. Likewise, read disturb can unintentionally increase the threshold voltage of a memory cell. Over time, the locations of the threshold voltage distributions may change. Such changes can increase the bit error rate, thereby increasing decoding time or even making decoding impossible. Changing the read reference voltages can help to mitigate such effects. Using ECC during the read process can fix errors and ambiguities. Note that in some embodiments, the threshold voltage distributions for a population of memory cells storing four bits of data per memory cell do not overlap and are separated from each other. The threshold voltage distributions of FIG. 5D will include read reference voltages and verify reference voltages, as discussed above.

When using four bits per memory cell, the memory can be programmed using the full sequence programming discussed above, or multi-pass programming processes known in the art. Each threshold voltage distribution (data state) of FIG. 5D corresponds to predetermined values for the set of data bits. The specific relationship between the data programmed into the memory cell and the threshold voltage

levels of the cell depends upon the data encoding scheme adopted for the cells. Table 3 provides an example of an encoding scheme for embodiments in which each bit of data of the four bits of data stored in a memory cell are in different logical pages, referred to as a lower page (LP), middle page (MP), an upper page (UP) and top page (TP).

TABLE 3

	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15
TP	1	1	1	1	1	0	0	0	0	0	1	1	0	0	0	1
UP	1	1	0	0	0	0	0	0	1	1	1	1	1	1	0	0
MP	1	1	1	0	0	0	0	1	1	0	0	0	0	1	1	1
LP	1	0	0	0	1	1	0	0	0	0	0	1	1	1	1	1

15

FIG. 6 is a flowchart describing one embodiment of a process for programming memory cells. For purposes of this document, the term program and programming are synonymous with write and writing. In one example embodiment, the process of FIG. 6 is performed for memory array 202 using the one or more control circuits (e.g., system control logic 260, column control circuitry 210, row control circuitry 220) discussed above. In one example embodiment, the process of FIG. 6 is performed by integrated memory assembly 207 using the one or more control circuits (e.g., system control logic 260, column control circuitry 210, row control circuitry 220) of control die 211 to program memory cells on memory die 201. The process includes multiple loops, each of which includes a program phase and a verify phase. The process of FIG. 6 is performed to implement the full sequence programming, as well as other programming schemes including multi-stage programming. When implementing multi-stage programming, the process of FIG. 6 is used to implement any/each stage of the multi-stage programming process.

Typically, the program voltage applied to the control gates (via a selected data word line) during a program operation is applied as a series of program pulses (e.g., voltage pulses). Between programming pulses are a set of verify pulses (e.g., voltage pulses) to perform verification. In many implementations, the magnitude of the program pulses is increased with each successive pulse by a predetermined step size. In step 602 of FIG. 6, the programming voltage signal (V_{pgm}) is initialized to the starting magnitude (e.g., ~12-16V or another suitable level) and a program counter PC maintained by state machine 262 is initialized at 1. In one embodiment, the group of memory cells selected to be programmed (referred to herein as the selected memory cells) are programmed concurrently and are all connected to the same word line (the selected word line). There will likely be other memory cells that are not selected for programming (unselected memory cells) that are also connected to the selected word line. That is, the selected word line will also be connected to memory cells that are supposed to be inhibited from programming. Additionally, as memory cells reach their intended target data state, they will be inhibited from further programming. Those NAND strings (e.g., unselected NAND strings) that include memory cells connected to the selected word line that are to be inhibited from programming have their channels boosted to inhibit programming. When a channel has a boosted voltage, the voltage differential between the channel and the word line is not large enough to cause programming. To assist in the boosting, in step 604 the control die will pre-charge channels of NAND strings that include memory cells connected to the selected word line that are to be inhibited from programming. In step 606,

NAND strings that include memory cells connected to the selected word line that are to be inhibited from programming have their channels boosted to inhibit programming. Such NAND strings are referred to herein as “unselected NAND strings.” In one embodiment, the unselected word lines receive one or more boosting voltages (e.g., ~7-11 volts) to

perform boosting schemes. A program inhibit voltage is applied to the bit lines coupled the unselected NAND string.

In step 608, a program voltage pulse of the programming voltage signal V_{pgm} is applied to the selected word line (the word line selected for programming). If a memory cell on a NAND string should be programmed, then the corresponding bit line is biased at a program enable voltage. In step 608, the program pulse is concurrently applied to all memory cells connected to the selected word line so that all of the memory cells connected to the selected word line are programmed concurrently (unless they are inhibited from programming). That is, they are programmed at the same time or during overlapping times (both of which are considered concurrent). In this manner all of the memory cells connected to the selected word line will concurrently have their threshold voltage change, unless they are inhibited from programming.

In step 610, program-verify is performed, which includes testing whether memory cells being programmed have successfully reached their target data state. Memory cells that have reached their target states are locked out from further programming by the control die. Step 610 includes performing verification of programming by sensing at one or more verify reference levels. In one embodiment, the verification process is performed by testing whether the threshold voltages of the memory cells selected for programming have reached the appropriate verify reference voltage. In step 610, a memory cell may be locked out after the memory cell has been verified (by a test of the V_t) that the memory cell has reached its target state.

In one embodiment of step 610, a smart verify technique is used such that the system only verifies a subset of data states during a program loop (steps 604-626). For example, the first program loop includes verifying for data state A (see FIG. 5C), depending on the result of the verify operation the second program loop may perform verify for data states A and B, depending on the result of the verify operation the third program loop may perform verify for data states B and C, and so on.

In step 616, the number of memory cells that have not yet reached their respective target threshold voltage distribution are counted. That is, the number of memory cells that have, so far, failed to reach their target state are counted. This counting can be done by state machine 262, memory controller 120, or another circuit. In one embodiment, there is one total count, which reflects the total number of memory cells currently being programmed that have failed the last verify step. In another embodiment, separate counts are kept for each data state.

In step 617, the system determines whether the verify operation in the latest performance of step 610 included

verifying for the last data state (e.g., data state G of FIG. 5C). If so, then in step 618, it is determined whether the count from step 616 is less than or equal to a predetermined limit. In one embodiment, the predetermined limit is the number of bits that can be corrected by error correction codes (ECC) during a read process for the page of memory cells. If the number of failed cells is less than or equal to the predetermined limit, then the programming process can stop and a status of "PASS" is reported in step 614. In this situation, enough memory cells programmed correctly such that the few remaining memory cells that have not been completely programmed can be corrected using ECC during the read process. In some embodiments, the predetermined limit used in step 618 is below the number of bits that can be corrected by error correction codes (ECC) during a read process to allow for future/additional errors. When programming less than all of the memory cells for a page, the predetermined limit can be a portion (pro-rata or not pro-rata) of the number of bits that can be corrected by ECC during a read process for the page of memory cells. In some embodiments, the limit is not predetermined. Instead, it changes based on the number of errors already counted for the page, the number of program-erase cycles performed or other criteria.

If in step 617 it was determined that the verify operation in the latest performance of step 610 did not include verifying for the last data state or in step 618 it was determined that the number of failed memory cells is not less than the predetermined limit, then in step 619 the data states that will be verified in the next performance of step 610 (in the next program loop) is adjusted as per the smart verify scheme discussed above. In step 620, the program counter PC is checked against the program limit value (PL). Examples of program limit values include 6, 12, 16, 19, 20 and 30; however, other values can be used. If the program counter PC is not less than the program limit value PL, then the program process is considered to have failed and a status of FAIL is reported in step 624. If the program counter PC is less than the program limit value PL, then the process continues at step 626 during which time the Program Counter PC is incremented by 1 and the programming voltage signal V_{pgm} is stepped up to the next magnitude. For example, the next pulse will have a magnitude greater than the previous pulse by a step size ΔV_{pgm} (e.g., a step size of 0.1-1.0 volts). After step 626, the process continues at step 604 and another program pulse is applied to the selected word line (by the control die) so that another program loop (steps 604-626) of the programming process of FIG. 6 is performed.

In one embodiment memory cells are erased prior to programming. Erasing is the process of changing the threshold voltage of one or more memory cells from a programmed data state to an erased data state. For example, changing the threshold voltage of one or more memory cells from state P to state E of FIG. 5A, from states A/B/C to state E of FIG. 5B, from states A-G to state Er of FIG. 5C or from states S1-S15 to state S0 of FIG. 5D.

One technique to erase memory cells in some memory devices is to bias a p-well (or other types of) substrate to a high voltage to charge up a NAND channel. An erase enable voltage (e.g., a low voltage) is applied to control gates of memory cells while the NAND channel is at a high voltage to erase the memory cells. Herein, this is referred to as p-well erase.

Another approach to erasing memory cells is to generate gate induced drain leakage ("GIDL") current to charge up the NAND string channel. An erase enable voltage is applied

to control gates of the memory cells, while maintaining the NAND string channel potential to erase the memory cells. Herein, this is referred to as GIDL erase. Both p-well erase and GIDL erase may be used to lower the threshold voltage (Vt) of memory cells.

In one embodiment, the GIDL current is generated by causing a drain-to-gate voltage at a GIDL generation transistor (e.g., transistors connected to SGDT0, SGDT1, SGDT2, SGSB0, SGSB1 and SGSB2). In some embodiments, a select gate (e.g., SGD or SGS) can be used as a GIDL generation transistor. A transistor drain-to-gate voltage that generates a GIDL current is referred to herein as a GIDL voltage. The GIDL current may result when the GIDL generation transistor drain voltage is significantly higher than the GIDL generation transistor control gate voltage. GIDL current is a result of carrier generation, i.e., electron-hole pair generation due to band-to-band tunneling and/or trap-assisted generation. In one embodiment, GIDL current may result in one type of carriers (also referred to a charge carriers), e.g., holes, predominantly moving into the NAND channel, thereby raising or changing the potential of the channel. The other type of carriers, e.g., electrons, are extracted from the channel, in the direction of a bit line or in the direction of a source line, by an electric field. During erase, the holes may tunnel from the channel to a charge storage region of the memory cells (e.g., to charge trapping layer 493) and recombine with electrons there, to lower the threshold voltage of the memory cells.

The GIDL current may be generated at either end (or both ends) of the NAND string. A first GIDL voltage may be created between two terminals of a GIDL generation transistor (e.g., connected to SGDT0, SGDT1, SGDT2) that is connected to or near a bit line to generate a first GIDL current. A second GIDL voltage may be created between two terminals of a GIDL generation transistor (e.g., SGSB0, SGSB1 and SGSB2) that is connected to or near a source line to generate a second GIDL current. Erasing based on GIDL current at only one end of the NAND string is referred to as a one-sided GIDL erase. Erasing based on GIDL current at both ends of the NAND string is referred to as a two-sided GIDL erase. The technology described herein can be used with one-sided GIDL erase and two-sided GIDL erase.

FIG. 7A depicts the programming signal V_{pgm} as a series of programming voltage pulses, such that one pulse of the programming signal V_{pgm} is applied at each performance of step 608 of FIG. 6. These programming voltage pulses are one example of doses of programming applied to a plurality of non-volatile memory cells being programmed. In one embodiment, the programming voltage pulses increase in voltage magnitude from pulse-to-pulse by a step size ΔV_{pgm} . In some embodiments, ΔV_{pgm} can change during a programming process. As described by FIG. 6, the system performs program-verification between the doses of programming (between or after programming voltage pulses), as depicted in FIGS. 7B and 7C. FIG. 7B, which illustrates an example in which program-verify is performed for one verify level, depicts two of the programming voltage pulses 702 and 704 of FIG. 7A. Between programming voltage pulses 702 and 704 is verify voltage pulse 710. In one embodiment, verify voltage pulse 710 has a magnitude of any of the verify reference voltages V_{vA} , V_{vB} , V_{vC} , V_{vD} , V_{vE} , V_{vF} , and V_{vG} (see FIG. 5C) and represents the system performing program-verify (step 610) between the doses of programming (successive iterations of step 608). In other embodiments, verify voltage pulse 710 has a magnitude of an intermediate voltage for a data state, as will be discussed

below. In some embodiments, between programming voltage pulses the system will perform program-verify for all data states, while in other embodiments, the system will perform program-verify for one data state at a time or a subset of data states.

FIG. 7C, which illustrates an example in which program-verify is performed for two verify levels, depicts two of the programming voltage pulses **702** and **704** of FIG. 7A. Between programming voltage pulses **702** and **704** are verify voltage pulses **710** and **712**. In one embodiment, verify voltage pulses **710** and **712** are for the same data state, while in other embodiments verify voltage pulses **710** and **712** are for different data states.

One means of verifying the programming (i.e. program-verify) is to test conduction at a specific compare point (e.g., the verify reference voltages VvA, VvB, VvC, VvD, VvE, VvF, and VvG of FIG. 5C). The memory cells that are verified to be sufficiently programmed are locked out, for example, by raising the bit line voltage from 0 to Vdd (e.g., 1.3 v) to stop the programming process for those memory cells. For example, FIG. 8A depicts graphs of threshold voltage (Vth) versus time and bit line voltage (Vbl) versus time. While a memory cell is receiving the program voltage Vpgm of FIG. 7A at its control gate, the threshold voltage of the memory cell increases. When the threshold voltage of the memory cell reaches the verify reference voltage Vvx (e.g. any of the verify reference voltages VvA, VvB, VvC, VvD, VvE, VvF, and VvG of FIG. 5C), then the bit line voltage is raised to Vinhibit (e.g. Vdd).

When programming and verifying as depicted in FIG. 8A, there is a tradeoff between speed of programming and precision of programming. The precision of programming is related to the distribution of threshold voltages of the programmed memory cells subsequent to the programming process. The tighter the threshold voltage distribution, the easier it is to unambiguously read the memory cells. The need for tight threshold voltage distributions is even more important with multi-level memory cells storing multiple bits of data because the read process needs to unambiguously distinguish between the different threshold voltage distributions (e.g., for data states Er-G). To obtain a tight threshold voltage distribution, a smaller step size (e.g., a smaller ΔV_{pgm}) can be used for the program voltage Vpgm. However, using a smaller step size slows down the programming process.

An improvement to the programming process is depicted in FIG. 8B, which can be referred to as a coarse/fine programming process. The process of FIG. 8B applies the program voltage signal Vpgm of FIG. 7A to the control gates of the memory cells to be programmed. Between the programming voltage pulses, program-verify operations are performed. If the threshold voltage of the memory cell being programmed is less than an intermediate verify reference voltage Vv_low (a lower verify voltage), the programming continues for that memory cell with the bit line voltage remaining low (e.g. 0 volts). This is the coarse phase of the coarse/fine programming process because the memory cells program faster. If the threshold voltage of the memory cell being programmed is higher than intermediate verify reference voltage Vv_low and lower than final verify reference voltage Vv_high (a higher verify voltage), then an intermediate bit line voltage (e.g. 0.5-1.0 volts) is applied. As a result of the intermediate bit line voltage, the channel voltage for the memory cell will increase and the programming of that memory cell will be slowed down because the shift in threshold voltage due to each programming voltage pulse will be reduced. This is the fine phase of the coarse/

fine programming process. The bit line will remain at the intermediate bit line voltage for a number of programming voltage pulses until the threshold voltage of the memory cell reaches the final verify reference voltage Vv_high. When the memory cell's threshold voltage reaches final verify reference voltage Vv_high, the bit line will be raised to inhibit further programming (e.g. by raising the bit line voltage to Vinhibit (e.g., Vdd, which can be 1-3 volts). Note that the final verify reference voltage Vv_high corresponds to the verify reference voltage for the data state that the memory cells are being programmed to (e.g. verify reference voltages VvA, VvB, VvC, VvD, VvE, VvF, and VvG of FIG. 5C). In one embodiment, the intermediate verify reference voltage Vv_low for a data state is 0.25 volts below the final verify reference voltage Vv_high for the data state. Other magnitudes of separation between Vv_low and Vv_high for a data state can also be used. In another embodiment, rather than raising the bit lines to the intermediate bit line voltage when the threshold voltage of the memory cell being programmed is higher than Vv_low and lower Vv_high, the system can reduce the size of ΔV_{pgm} to slow down the speed of programming during the fine phase.

Looking back at FIG. 7C, verify voltage pulse **710** can be set for Vv_low and verify voltage pulse **712** can be set for Vv_high. Thus, the memory system performs two sensing operations: one sensing operation for Vv_low (pulse **710**) and one sensing operation for Vv_high (pulse **712**). After each sensing operation, the memory system performs a scan of the results (e.g., check the result of the sensing for each bit line or each memory cells being programmed). In one embodiment, when starting to perform program verify for a data state (e.g., data states A-G of FIG. 5C), the memory system will start by only sensing at the Vv_low verify level for the data state according to the example of FIG. 7B. Once one or more memory cells have reached the Vv_low verify level for the data state, then the memory system will sense at both Vv_low and Vv_high for the data state according to the example of FIG. 7C.

Using the approach of FIG. 8B results in the programmed threshold voltage distributions being narrower than the process of FIG. 8A because the shift per program pulse of the threshold voltage is reduced once the threshold voltage is close to the target value (e.g. when the threshold voltage is above Vv_low and below Vv_high). However, the approach of FIG. 8B slows down the programming process because each data state being program-verified now requires two sensing operations (e.g., one sensing operation at Vv_low and one sensing operation at Vv_high). To reduce the time needed for program-verify, a portion of the process for verifying at Vv_low is performed concurrently with a portion of the process for verifying at Vv_high. More details will be discussed below with respect to FIG. 12.

The programming, verifying/sensing and erasing processes discussed above all require that various voltages be applied to the word lines of the selected block(s) of memory cells. FIG. 9 is a block diagram that depicts charge pumps, control gate drivers, a switch network, and word line switches that are used to apply those voltages to the word lines. FIG. 9 shows a set of charge pumps **902** providing multiple different sources of voltages to multiple sets of control gate drivers including a first set of control gate drivers **904**, a second set of control gate drivers **906**, a third set of control gate drivers **908**, . . . a X set of control gate drivers **910**. Charge pumps **902** include multiple charge pumps that supply the different voltages needed to perform memory operations. In some embodiments, voltage generators can also generate some of the different voltages needed

25

to perform memory operations. In one embodiment, there is a separate set of one or more charge pumps or voltage generator for each voltage needed to be applied to the word lines. In other embodiments, some charge pumps and/or voltage generators can be the source of multiple voltages by using voltage dividers or other circuits.

In one embodiment, first set of control gate drivers **904** includes multiple control gate drivers for supplying the necessary voltages to data word lines (e.g., WL-WL**239**) in order to perform programming, erasing and reading. The output of the first set of control gate drivers **904**, CGout_A (which includes one separate output signal for each control gate driver), is provided (connected) to switch network **920**. First set of control gate drivers **904** are connected to the data word lines via switch network **920**. In one embodiment, all of the control gate drivers of the first set of control gate drivers **904** have the same structure, receive the same inputs (source of voltage) and provide the same set of voltage outputs.

In one embodiment, second set of control gate drivers **906** includes multiple control gate drivers for supplying the necessary voltages to dummy word lines in order to perform programming, erasing and reading. The output of the second set of control gate drivers **906**, CGout_B (which includes one separate output signal for each control gate driver), is provided (connected) to switch network **920**. The second set of control gate drivers **906** are connected to the dummy word lines via switch network **920**. In one embodiment, all of the control gate drivers of the second set of control gate drivers **906** have the same structure, receive the same inputs (sources of voltage) and provide the same set of voltage outputs.

In one embodiment, third set of control gate drivers **908** includes multiple control gate drivers for supplying the necessary voltages to select lines (e.g., SGD and SGS) in order to perform programming, erasing and reading. The output of the third set of control gate drivers **908**, CGout_C (which includes one separate output signal for each control gate driver), is provided (connected) to switch network **920**. The third set of control gate drivers **908** are connected to the select lines via switch network **920**. In one embodiment, all of the control gate drivers of the third set of control gate drivers **908** have the same structure, receive the same inputs (sources of voltage) and provide the same set of voltage outputs.

FIG. 9 shows word lines WL0-WL**161** for two example blocks (Block A and Block B). Each word line WL0-WL**161** for each block is connected to an output of switch network **920** via a word line switch transistor (e.g., **940-966**). For example, word line WL0 of block A is connected to the output of word line switch transistor **966**, WL**159** of block A is connected to the output of word line switch transistor **964**, WL**160** of block A is connected to the output of word line switch transistor **962**, WL**161** of block A is connected to the output of word line switch transistor **960**, WL0 of block B is connected to the output of word line switch transistor **946**, WL**159** of block B is connected to the output of word line switch transistor **944**, WL**160** of block B is connected to the output of word line switch transistor **942**, and WL**161** of block B is connected to the output of word line switch transistor **940**. The inputs to the word line switches are connected to outputs of switch network **920**. In one embodiment, switch network **920** includes a plurality of high voltage switches (e.g., transistors) to connect the control gate drivers to the appropriate word lines by routing the output voltage from the control gate drivers to the appropriate word line switch transistors **940-966** connected to the

26

word lines WL0-WL**161** of each block. Note that in some embodiments, multiple word lines can share a single control gate driver. The switch network is also connected to the select lines SGS and SGD (which are omitted from FIG. 9 to make FIG. 9 easier to read).

FIG. 9 also depicts voltage control circuit **903**, which is connected to charge pumps **902**. Voltage control circuit **903** controls the output of the charge pumps. In one embodiment, voltage control circuit **903** receives a digital signal from the state machine or other processor that indicates the desired charge pump output, and voltage control circuit **903** includes a digital to analog conversion circuit that outputs a signal to the charge pumps that indicates the requested voltage signal to be output by the charge pumps.

FIG. 10 is a timing diagram depicting the behavior of the following signals during a program-verify operation: BL(sel), BL(unsel), SGD(sel), SGD(unsel), WLunsel, WLn, SGS and SL. The signal BL(sel) is the voltage applied to bit lines connected to NAND strings having memory cells selected for sensing/reading. The signal BL(unsel) is the voltage applied to bit lines connected to NAND strings that do not have any memory cells selected for sensing/reading. In some embodiments, all bit lines will be selected for sensing/reading. The signal SGD(sel) is the voltage applied to the drain side select (SGD) lines (e.g., SGD0, SGD1 and SGD2 connected together for one region) for the region (e.g., regions **420**, **430**, **440**, **450**, **460** and **470**) selected for sensing/reading. The signal SGD(unsel) is the voltage applied to the drain side select (SGD) lines (e.g., SGD0, SGD1 and SGD2 connected together for one region) for the region (e.g., regions **420**, **430**, **440**, **450**, **460** and **470**) not selected for sensing/reading. The signal SGS is the voltage applied to the source side select (SGS) lines (e.g., SGS0, SGS1 and SGS2 connected together). The signal SL is the voltage applied to the source line. The signal WLn is the voltage applied to the word line selected for programming and program-verify (i.e. the word line connected to the memory cells selected for programming). The signal WLunsel is the voltage applied to the word lines not selected to be programmed.

At time **t0** of FIG. 10, all signals depicted in FIG. 10 are at Vss (ground or 0 volts). At time **t1**, BL(sel) is raised to Vbl (e.g., 0.5-1.5 v) and WLunsel is raised to Vread (e.g., 6-8 volts). Vread is an example of an overdrive voltage because it is high enough to turn on the memory cell regardless of which data state the memory cell has been programmed to. Also at time **t1**, SGD(sel) is raised to Vsg (e.g., 3.5-6 v), an example of a sense enabling voltage, in two steps such that it reaches Vdd at **t2** and Vsg at **t3**. At **t1**, a voltage spike is applied to WLn and, subsequent to the voltage on WLn settling down, WLn is raised to Vcgv (e.g., one of the verify reference voltages VvA, VvB, VvC, VvD, VvE, VvF, and VvG of FIG. 5C). At time **t4**, SGS is raised to Vsg, which provides a path for the bit line voltage to dissipate. If Vcgv is greater than the threshold voltage of the selected memory cells, then the selected memory cells will conduct current and the bit line voltage will dissipate via the source line, as depicted by curve **994**. If Vcgv is not greater than the threshold voltage of the selected memory cells, then the selected memory cells will not conduct current and the bit line voltage will not dissipate via the source line, as depicted by curve **992**. The sense amplifiers will sense whether the selected memory cell conducted or not at time **t5**. At time **t6**, BL(sel) is lowered to Vss. At time **t7**, SGD(sel), WLunsel, WLn, and SGS are lowered to Vss. When sensing at **t5**, the results of the sensing are stored in a latch at the respective sense amplifier. Afterwards, the system (e.g., control circuit)

scans all of the latches of the sense amplifiers to determine which memory cells conducted and which did not conduct.

FIG. 11 depicts two graphs horizontally aligned to each other. The top graph describes the voltages applied to the unselected word lines (WL_{unsel}) and the bottom graph depicts current (ICC) drawn by the memory system. The horizontal axis for both graphs represents time and is divided into two time periods: R1 and R2. The time period R1 corresponds to the time period between t1 and t2 of FIG. 10. The time period R2 corresponds to the time period between t2 and t3 of FIG. 10. The top graph includes four curves: 1102, 1104, 1106 and 1108. Curves 1102 and 1106 represent two example option for signal 905 that is output from voltage control circuit 903 (see FIG. 9) and input to charge pumps 902. Curve 1102 is a control signal that configures charge pumps 902 to ramp up the word line voltage to V_{read} as fast as possible. Curve 1106 is a control signal that configures charge pumps 902 to increase the word line voltage at a slower rate (slower ramp). Curve 1106 can be referred to as a step signal as it raises the voltages in a series of steps. Curve 1104 is the output of the charge pumps 902 that are supplied to the unselected word lines during program-verify in response to voltage control circuit 903 inputting curve 1102 to charge pumps 902. Curve 1108 is the output of the charge pumps 902 that are supplied to the unselected word lines during program-verify in response to voltage control circuit 903 inputting curve 1106 to charge pumps 902. As can be seen, curve 1108 has a more gentle increase in voltage magnitude than curve 1104. Curve 1108 shows the word line voltage increasing in steps.

The bottom graph shows three curves: 1120, 1122 and 1124. Curve 1120 shows a moving average of the current ICC drawn in the memory die when curve 1104 is the output of the charge pumps 902 in response to voltage control circuit 903 inputting curve 1102 to charge pumps 902. Curve 1122 shows a moving average of the current ICC drawn in the memory die when curve 1108 is the output of the charge pumps 902 in response to voltage control circuit 903 inputting curve 1106 to charge pumps 902. Curve 1124 (comprising a series of bell curves) represents the raw current (rather than moving average) when curve 1108 is the output of the charge pumps 902 in response to voltage control circuit 903 inputting curve 1106 to charge pumps 902. As can be seen from FIG. 11, slowing down the ramping up of the word lines voltage (e.g., increasing the word line voltage in steps rather than a single high slope ramp) results in a lower current ICC.

As described above, in the embodiments of the memory that use the coarse/fine programming of FIG. 8B, in order to reduce the time needed for program-verify, a portion of the process for verifying at V_{v_low} is performed concurrently with a portion of the process for verifying at V_{v_high}. For example, the scanning of the results of testing for V_{v_low} that are stored in the latches of the sense amplifiers can be performed during R1 of the verify operation for V_{v_high}, as depicted in FIG. 12.

FIG. 12 depicts two graphs horizontally aligned to each other. The top graph describes the voltages applied to the unselected word lines (WL_{unsel}) and the bottom graph depicts current (ICC) drawn by the memory system. The horizontal axis for both graphs represents time and is divided into two time periods: R1 and R2. The top graph includes two curves: 1202 and 1204. Curves 1202 is a step signal that is output from voltage control circuit 903 and input to charge pumps 902 in order to slowly increase word line voltage, similar to curve 1106 of FIG. 11. While the word line voltages are ramping up to V_{read}, the scanning of

the results of the previous sensing (e.g., testing for V_{v_low}) that are stored in the latches of the sense amplifiers can be performed during R1 of the current sensing operation. In prior systems, the time period R1 is elongated until the scanning finished; therefore, the charge pumps 902 will ignore the specific steps of input signal 905 (curve 1202) and ramp up the word line voltage independently of the steps of curve 1202. Once the scanning completes, then the charge pumps 902 will follow input signal 905 (curve 1202) and ramp up the word line voltage in response to the steps of curve 1202. However, the charge pumps cannot immediately follow curve 1202, which pulls down the output of the charge pumps, giving rise to a large peak current, as depicted in the bottom graph of FIG. 12. More specifically, as the word line voltages ramp up during R1, the control gates of the memory cells couple to the channel (SGD rises to V_{dd} so the channel is weakly conducting). When the scan ends, the charge pumps 902 try to catch up to curve 1202, but SGD starts ramping up to V_{sg} (see FIG. 10), so the channel of the NAND strings start to conduct. As a result, charge leaks out of the channel to the bit line and the channel potential goes down. This, in turn, pulls down the output of charge pumps 902 until they slowly catch up to curve 1202. The large mismatch between curve 1202 (output from voltage control circuit 903) and the output of charge pumps 902 gives rise to the large ICC current.

To reduce the above-described peak current issue during program-verify, it is proposed to separate the scanning of the results of previous sensing that are stored in the latches of the sense amplifiers from the time period R1 and have the charge pumps follow the input signal 905 (e.g., curve 1202) from the start of the voltage ramp up. That is, the scanning of the results of previous sensing can still be performed during (concurrently with) R1; however, the time period R1 will not be elongated until the scanning finished. Rather R1 will be a set time period that ends based on a configuration parameter (default value or supplied by a user) rather than being based on completion of the scanning of previous results. Thus, it is likely (but not required) that the time period R1 will end before the scanning ends. Thus, while scanning the results of a previous program-verify operation, the voltages on the select line (e.g., SGD) for the next program-verify operation will ramp up without waiting for the scan to complete and the voltages on unselected word lines for the next program-verify operation will ramp up and follow the step signal 1202 without waiting for the scan to complete. This concept is depicted in FIG. 13.

FIG. 13 includes two sets of graphs 1300 and 1302. Each set of graphs include a top graph showing voltage applied to the select line SGD and a bottom graph (horizontally aligned with the top graph) showing voltage applied to the unselected word lines (WL_{unsel}). For each graph, the horizontal axis is time. The set of graphs 1300 on the left show the embodiment of FIG. 12, where R1 is elongated until the scan is finished. The set of graphs 1302 on the right show the proposed system that separates the scanning of the results of previous sensing that are stored in the latches of the sense amplifiers from the time period R1, and have the charge pumps follow the input signal 905 (e.g., curve 1202) from the start of the word line voltage ramp up. Thus, the bottom right graph shows the output signal 1314 from charge pumps 902 following the step signal 1312 input to the charge pumps 902 from voltage control circuit 903. In other words, output signal 1314 from charge pumps 902 ramps up (resulting in ramping up the voltages on unselected word lines) for the next program-verify operation following step signal 1312 while the memory system is performing the scan 1320 of the

results of a previous program-verify operation and without waiting for the scan 1320 to complete. As depicted in graphs 1302, the voltages on unselected word lines are ramped up for the next program-verify operation by using the charge pumps 902 to ramp up voltage on unselected word lines to an overdrive voltage V_{read} while performing the scan 1320 of the results of the previous program-verify operation and without waiting for the scan 1320 to complete in response to the step signal 1312 so that voltage applied to the unselected word lines 1314 rises in steps (as depicted by curve 1314). Although FIG. 13 shows a curve 1322 for SGD, in some embodiments that same curve 1322 can be used for the voltage applied to SGS. The curves 1312, 1314 and 1322 of FIG. 13 are examples of the curves depicted in FIG. 10 during R1 and R2.

In some cases, as depicted in FIG. 13, the voltages on the unselected word lines reach the overdrive voltage V_{read} after the scan 1320 of the results of the previous program-verify operation completes. Additionally, as depicted in FIG. 13, the voltage on the select line SGD reaches V_{dd} before scan 1320 completes and reaches the sensing enable voltage (V_{sg}) after scan 1320 completes.

FIG. 14 is a flow chart describing one embodiment of a process for programming, including performing program-verify, based on the embodiment of graphs 1302. Thus, the process of FIG. 14 includes separating the scanning of the results of previous sensing that are stored in the latches of the sense amplifiers from the time period R1 and having the charge pumps follow the input signal 905 (e.g., curve 1312) from the start of the voltage ramp up. The process of FIG. 14 can be performed by any one of the one or more control circuits discussed above. The process of FIG. 14 can be performed entirely by a control circuit on memory die 200 (see FIG. 2A) or entirely by a control circuit on integrated memory assembly 207 (see FIG. 2B), rather than by memory controller 120. In one example, the process of FIG. 14 is performed by or at the direction of state machine 262, using other components of System Control Logic 260, Column Control Circuitry 210 and Row Control Circuitry 220. In another embodiment, the process of FIG. 14 is performed by or at the direction of memory controller 120. The process of FIG. 14, which can be performed on a memory implementing any of the structures depicted in FIGS. 1-4F and 9, is based on and/or uses the process of FIG. 10.

In step 1402 of FIG. 14, memory cells are programmed by applying doses of programming (e.g., the program voltage pulses of step 608 and FIG. 7A). In step 1404, program-verify is performed between doses of programming (e.g., step 610). One example implementation of step 1404 comprises ramping up voltage on a select line (e.g. SGD) for a next program-verify operation while performing a scan of the results of a previous program-verify operation and without waiting for the scan to complete (step 1420), and ramping up the voltages on unselected word lines (e.g., WL_{unsel}) for the next program-verify operation following a step signal (e.g., 1312) while performing a scan (e.g., scan 1320) of the results of a previous program-verify operation and without waiting for the scan to complete (step 1422). One example of step 1420 is curve 1322. One example of step 1422 is curve 1314. In one embodiment, the previous program-verify operation is from V_{v_low} and the next program-verify operation is for V_{v_high} of the same data state. This speeds up the program-verify process by scanning the results for V_{v_low} while starting the program-verify operation for V_{v_high} . Therefore, in this embodiment, the control circuit is configured to perform program-verify between doses of programming by ramping up voltage on

the select line in order to test for the higher verify level (V_{v_high}) while performing the scan of the results of the testing for the lower verify level (V_{v_low}) and without waiting for the scan to complete and ramping up the voltages on unselected word lines in order to test for the higher verify level while performing a scan of the results of the testing for the lower verify level and without waiting for the scan to complete. In other embodiments, the previous program-verify operation and the next program-verify operation could be for different data states or could be for the same verify reference voltage.

FIG. 15 is a flow chart describing an embodiment of a process for programming, including performing program-verify, based on the embodiment of graphs 1302. Thus, the process of FIG. 15 includes separating the scanning of the results of previous sensing that are stored in the latches of the sense amplifiers from the time period R1 and having the charge pumps follow the input signal 905 (e.g., curve 1312) from the start of the voltage ramp up. The process of FIG. 15 is an example implementation of the process of FIG. 14.

The process of FIG. 15 can be performed by any one of the one or more control circuits discussed above. The process of FIG. 15 can be performed entirely by a control circuit on memory die 200 (see FIG. 2A) or entirely by a control circuit on integrated memory assembly 207 (see FIG. 2B), rather than by memory controller 120. In one example, the process of FIG. 15 is performed by or at the direction of state machine 262, using other components of System Control Logic 260, Column Control Circuitry 210 and Row Control Circuitry 220. In another embodiment, the process of FIG. 15 is performed by or at the direction of memory controller 120. The process of FIG. 15, which can be performed on a memory implementing any of the structures depicted in FIGS. 1-4F and 9, is based on and/or uses the process of FIG. 10.

In step 1502 of FIG. 15, memory cells are programmed by applying doses of programming (e.g., the program voltage pulses of step 608 and FIG. 7A). In step 1504, the control circuit verifies status of programming after applying the doses of programming (e.g., step 610).

One example implementation of step 1504 comprises verifying status for a lower verify level (e.g., V_{v_low}) for a data state (step 1512) and verifying status for a higher verify level (e.g., V_{v_high}) for the same data state (step 1514). The data state can be any of the programmed data states discussed above with respect to FIGS. 5A-5D.

One example implementation of step 1512 comprises performing a first sensing of memory cells to determine which memory cells have threshold voltage higher than the lower verify level, after applying a dose of programming, in step 1520. The results of the first sensing are scanned (see e.g., scan 1320) in step 1522. In one embodiment, the scanning comprises checking the value stored in a latch for each sense amplifier.

One example implementation of step 1514 comprises implementing the process of FIG. 10 including ramping up voltage on the select line (e.g., SGD) to a sensing enable voltage (e.g., V_{sg}) concurrently with, and without waiting for the completion of, the scanning results of the first sensing (see scan 1320) in step 1530; applying a step signal (e.g., 1312) to control a voltage source (e.g., charge pumps 902) connected to unselected word lines (WL_{unsel}) in step 1532; using the voltage source to ramp up voltage on unselected word lines to an overdrive voltage (e.g., V_{read}) concurrently with, and without waiting for the completion of, the scanning results of the first sensing (see scan 1320), in response to the step signal so that voltage applied to the

31

unselected word lines rises in steps (see curve **1314**) in step **1534**; applying a verify compare voltage (V_{cgv}) to a selected word line (WLn) in step **1542** (see FIG. **10**); and sensing memory cells connected to the selected word line in step **1544** (e.g., at $t5$ of FIG. **10**).

FIG. **16** is a flow chart describing an embodiment of a process for programming, including performing program-verify, based on the embodiment of graphs **1302**. Thus, the process of FIG. **16** includes separating the scanning of the results of previous sensing that are stored in the latches of the sense amplifiers from the time period **R1** and have the charge pumps follow the input signal **905** (e.g., curve **1202**) from the start of the voltage ramp up. The process of FIG. **16** is an example implementation of the process of FIG. **14** and/or the process of FIG. **15**.

The process of FIG. **16** can be performed by any one of the one or more control circuits discussed above. The process of FIG. **16** can be performed entirely by a control circuit on memory die **200** (see FIG. **2A**) or entirely by a control circuit on integrated memory assembly **207** (see FIG. **2B**), rather than by memory controller **120**. In one example, the process of FIG. **16** is performed by or at the direction of state machine **262**, using other components of System Control Logic **260**, Column Control Circuitry **210** and Row Control Circuitry **220**. In another embodiment, the process of FIG. **16** is performed by or at the direction of memory controller **120**. The process of FIG. **16**, which can be performed on a memory implementing any of the structures depicted in FIGS. **1-4F** and **9**, is based on and/or uses the process of FIG. **10**.

Step **1602** of FIG. **16** includes applying doses of programming to a set of non-volatile memory cells connected to a selected word line WLn (e.g., the program voltage pulses of step **608** and FIG. **7A**). Step **1604** of FIG. **16** includes performing program-verify between the doses of programming including testing for a lower verify level for a data state and testing for a higher verify level for the data state (see step **610**, FIG. **7C** and FIG. **8B**).

FIG. **17** is a flow chart describing an embodiment of a process for testing for a higher verify level for the data state. Thus, the process of FIG. **17** is performed as part of step **1604** of FIG. **16**. The process of FIG. **17** can be performed by any one of the one or more control circuits discussed above. The process of FIG. **17** can be performed entirely by a control circuit on memory die **200** (see FIG. **2A**) or entirely by a control circuit on integrated memory assembly **207** (see FIG. **2B**), rather than by memory controller **120**. In one example, the process of FIG. **17** is performed by or at the direction of state machine **262**, using other components of System Control Logic **260**, Column Control Circuitry **210** and Row Control Circuitry **220**. In another embodiment, the process of FIG. **17** is performed by or at the direction of memory controller **120**. The process of FIG. **17**, which can be performed on a memory implementing any of the structures depicted in FIGS. **1-4F** and **9**, is based on and/or uses the process of FIG. **10**.

In step **1702** of FIG. **17**, voltages on a select line (e.g., SGD) are ramped up (e.g., see curve **1322**) in order to test for the higher verify level while performing a scan (e.g., scan **1320**) of the results of the testing for the lower verify level and without waiting for the scan to complete. In step **1704**, the voltages on unselected word lines are ramped up (see e.g., curve **1314**) in order to test for the higher verify level (e.g., V_{v_high} of FIG. **8B**) while performing the scan (e.g., scan **1320**) of the results of the testing for the lower verify level (e.g., V_{v_low} of FIG. **8B**) and without waiting for the scan to complete. In step **1706**, a step signal (e.g.,

32

curve **1312**) is applied to control a voltage source (e.g., charge pumps **902**) connected to unselected word lines so that the ramping up the voltages on unselected word lines comprises using the voltage source to ramp up voltage on unselected word lines to an overdrive voltage concurrently with, and without waiting for the completion of, the scan of the results of the testing for the lower verify level in response to the step signal so that voltage applied to the unselected word lines rises in steps (see e.g., curve **1314**). In some embodiments, step **1706** is performed concurrently with or before steps **1702** and **1704**. Steps **1702-1706** are performed during $R1$ and $R2$ (e.g., between $t1$ - $t3$ of FIG. **10**). In step **1708**, a verify compare voltage V_{cgv} is applied to a selected word line WLn (see FIG. **10**). In step **1710**, the memory cells connected to the selected word line WLn are sensed (e.g., at time $t5$ of FIG. **10**) using the sense amplifiers. In step **1712**, the control circuit will slow down programming in response to memory cells having threshold voltages greater than the lower verify level for the data state; for example, by raising the bit line voltage (see e.g., FIG. **8B**) or decreasing ΔV_{pgm} .

A memory system has been proposed that includes an efficient and low current program-verify operation.

One embodiment includes a non-volatile storage apparatus comprising: a plurality of non-volatile memory cells; a plurality of word lines connected to the memory cells; a first select line connected to the memory cells; and a control circuit connected to the memory cells and word lines as well as the first select line. The control circuit is configured to program the memory cells by applying doses of programming and perform program-verify between doses of programming. The control circuit is configured to perform program-verify between doses of programming by: ramping up voltage on the first select line for a next program-verify operation while performing a scan of the results of a previous program-verify operation and without waiting for the scan to complete; and ramping up the voltages on unselected word lines for the next program-verify operation following a step signal while performing a scan of the results of a previous program-verify operation and without waiting for the scan to complete.

In one example implementation, the control circuit is further configured to perform program-verify between doses of programming by testing for a lower verify level for a data state and testing for a higher verify level for the data state; and the previous program-verify operation comprises the testing for the lower verify level for the data state and the next program-verify operation comprises the testing for the higher verify level for the data.

In one example implementation, the control circuit is further configured to perform program-verify between doses of programming by testing for a lower verify level for a data state and testing for a higher verify level for the data state; and the previous program-verify operation comprises the testing for the lower verify level for the data state and the next program-verify operation comprises the testing for the higher verify level for the data state. The control circuit is configured to perform program-verify between doses of programming by: ramping up voltage on the first select line in order to test for the higher verify level while performing the scan of the results of the testing for the lower verify level and without waiting for the scan to complete; and ramping up the voltages on unselected word lines in order to test for the higher verify level while performing a scan of the results of the testing for the lower verify level and without waiting for the scan to complete.

33

In one example implementation, the control circuit is further configured to perform program-verify between doses of programming by testing whether memory cells connected to a selected word line have threshold voltages greater than a lower verify level for a data state and testing whether memory cells connected to the selected word line have threshold voltages greater than a higher verify level for the data state between doses of programming; the previous program-verify operation comprises the testing whether memory cells connected to the selected word line have threshold voltages greater than the lower verify level for the data state; the testing whether memory cells connected to the selected word line have threshold voltages greater than a lower verify level for the data state includes the scan of the results; the next program-verify operation comprises the testing whether memory cells connected to the selected word line have threshold voltages greater than the higher verify level for the data state. Additionally, the control circuit is further configured to slow down programming for memory cells having threshold voltages greater than the lower verify level and less than the higher verify level.

In one example implementation, during program-verify between doses of programming, the control circuit is further configured to apply the step signal to control a voltage source connected to unselected word lines; and the control circuit is configured to ramp up the voltages on unselected word lines for the next program-verify operation by using the voltage source to ramp up voltage on unselected word lines to an overdrive voltage while performing the scan of the results of the previous program-verify operation and without waiting for the scan to complete in response to the step signal so that voltage applied to the unselected word lines rises in steps. Additionally, in some cases, the control circuit is further configured to perform program-verify between doses of programming by testing for a lower verify level for a data state and testing for a higher verify level for the data state; the previous program-verify operation comprises the testing for the lower verify level for the data state and the next program-verify operation comprises the testing for the higher verify level for the data; during program-verify between doses of programming, the control circuit is further configured to apply a verify compare voltage to a selected word line and sense memory cells connected to the selected word line; and the control circuit is further configured to slow down programming for memory cells having threshold voltages greater than the lower verify level and less than the higher verify level.

In one example implementation, the ramping up voltage on the first select line comprises ramping up voltage on the first select line to a sensing enable voltage while performing the scan of the results of the previous program-verify operation and without waiting for the scan to complete, the voltage on the first select line reaches the sensing enable voltage after the scan of the results of the previous program-verify operation completes; and ramping up the voltages on unselected word lines comprises ramping up the unselected word lines to an overdrive voltage while performing the scan of the results of the previous program-verify operation and without waiting for the scan to complete, the voltages on the unselected word lines reach the overdrive voltage after the scan of the results of the previous program-verify operation completes.

In one example implementation, the doses of programming are voltage pulses applied to a selected word line, the voltage pulses increase in voltage magnitude from pulse-to-pulse.

34

In one example implementation, the plurality of non-volatile memory cells are arranged in NAND strings and the plurality of word lines and the select line are connected to each of the NAND strings.

In one example implementation, the plurality of non-volatile memory cells are arranged in NAND strings and the plurality of word lines; the select line are connected to each of the NAND strings; a second select line connected to each of the NAND strings, the first select line is connected to each of the NAND strings at a first end of the NAND strings, the second select line is connected to each of the NAND strings at a second end of the NAND strings; a plurality of bit lines connected to the NAND strings at the first end of the NAND strings; and a source line connected to the NAND strings at the second end of the NAND strings, the control circuit is further configured to apply a verify compare voltage to a selected word line and sense memory cells connected to the selected word line as part of the performing program-verify between doses of programming.

One embodiment includes a method, comprising: applying doses of programming to a set of non-volatile memory cells connected to a selected word line; and performing program-verify between the doses of programming including testing for a lower verify level for a data state and testing for a higher verify level for the data state. The testing for a higher verify level for the data state comprises: ramping up voltages on a select line in order to test for the higher verify level while performing a scan of the results of the testing for the lower verify level and without waiting for the scan to complete; and ramping up the voltages on unselected word lines in order to test for the higher verify level while performing the scan of the results of the testing for the lower verify level and without waiting for the scan to complete.

In one example implementation, the process includes slowing down programming in response to memory cells having threshold voltages greater than the lower verify level for the data state.

In one example implementation, the testing for the higher verify level further comprises: applying a step signal to control a voltage source connected to unselected word lines; the ramping up the voltages on unselected word lines comprises using the voltage source to ramp up voltage on unselected word lines to an overdrive voltage concurrently with, and without waiting for the completion of, the scan of the results of the testing for the lower verify level in response to the step signal so that voltage applied to the unselected word lines rises in steps.

In one example implementation, the ramping up the voltages on unselected word lines is done in steps.

In one example implementation, the testing for the higher verify level for the data state further comprises: applying a verify compare voltage to a selected word line; and sensing memory cells connected to the selected word line.

In one example implementation, the ramping up voltages on the select line raises the select line first to a positive supply voltage and then to a sensing enable voltage; the select line reaches the positive supply voltage before the scan of the results of the testing for the lower verify level completes; and the select line reaches the sensing enable voltage after the scan of the results of the testing for the lower verify level completes.

In one example implementation, the ramping up voltages on unselected word lines raises the unselected word lines to an overdrive voltage; the ramping up voltages on unselected word lines starts before the scan of the results of the testing for the lower verify level completes; and the unselected

35

word lines reach the overdrive voltage after the scan of the results of the testing for the lower verify level completes.

One embodiment includes a non-volatile storage apparatus comprising: a plurality of non-volatile memory cells; a plurality of word lines connected to the memory cells; a select line connected to the memory cells; and a control circuit connected to the memory cells and word lines as well as the select line, the control circuit is configured to program the memory cells by applying a dose of programming and verify status of programming after applying the dose of programming, the verifying status of programming comprises verifying status for a lower verify level for a data state and verifying status for a higher verify level for the data state after applying the dose of programming, the verifying status for the lower verify level for the data state comprises a first sensing of memory cells to determine which memory cells have threshold voltage higher than the lower verify level and scanning results of the first sensing. The verifying status for the higher verify level for the data state comprises: ramping up voltage on a select line to a sensing enable voltage concurrently with, and without waiting for the completion of, the scanning results of the first sensing; applying a step signal to control a voltage source connected to unselected word lines; using the voltage source to ramp up voltage on unselected word lines to an overdrive voltage concurrently with, and without waiting for the completion of, the scanning results of the first sensing, in response to the step signal so that voltage applied to the unselected word lines rises in steps; applying a verify compare voltage to a selected word line; and sensing memory cells connected to the selected word line. In one example implementation, the plurality of non-volatile memory cells are arranged in NAND strings; and the plurality of word lines and the select line are connected to each of the NAND strings.

For purposes of this document, reference in the specification to “an embodiment,” “one embodiment,” “some embodiments,” or “another embodiment” may be used to describe different embodiments or the same embodiment.

For purposes of this document, a connection may be a direct connection or an indirect connection (e.g., via one or more other parts). In some cases, when an element is referred to as being connected or coupled to another element, the element may be directly connected to the other element or indirectly connected to the other element via one or more intervening elements. When an element is referred to as being directly connected to another element, then there are no intervening elements between the element and the other element. Two devices are “in communication” if they are directly or indirectly connected so that they can communicate electronic signals between them.

For purposes of this document, the term “based on” may be read as “based at least in part on.”

For purposes of this document, without additional context, use of numerical terms such as a “first” object, a “second” object, and a “third” object may not imply an ordering of objects, but may instead be used for identification purposes to identify different objects.

For purposes of this document, the term “set” of objects may refer to a “set” of one or more of the objects.

The foregoing detailed description has been presented for purposes of illustration and description. It is not intended to be exhaustive or to limit to the precise form disclosed. Many modifications and variations are possible in light of the above teaching. The described embodiments were chosen in order to best explain the principles of the proposed technology and its practical application, to thereby enable others skilled in the art to best utilize it in various embodiments and

36

with various modifications as are suited to the particular use contemplated. It is intended that the scope be defined by the claims appended hereto.

What is claimed is:

1. A non-volatile storage apparatus, comprising:
a plurality of non-volatile memory cells;
a plurality of word lines connected to the memory cells;
a first select line connected to the memory cells; and
a control circuit connected to the memory cells and word lines as well as the first select line, the control circuit is configured to program the memory cells by applying doses of programming and perform program-verify between doses of programming;

the control circuit is configured to perform program-verify between doses of programming by:

ramping up voltage on the first select line for a next program-verify operation while performing a scan of the results of a previous program-verify operation and without waiting for the scan to complete; and
ramping up the voltages on unselected word lines for the next program-verify operation following a step signal while performing a scan of the results of a previous program-verify operation and without waiting for the scan to complete.

2. The non-volatile storage apparatus of claim 1, wherein:
the control circuit is further configured to perform program-verify between doses of programming by testing for a lower verify level for a data state and testing for a higher verify level for the data state; and
the previous program-verify operation comprises the testing for the lower verify level for the data state and the next program-verify operation comprises the testing for the higher verify level for the data.

3. The non-volatile storage apparatus of claim 1, wherein:
the control circuit is further configured to perform program-verify between doses of programming by testing for a lower verify level for a data state and testing for a higher verify level for the data state; and
the previous program-verify operation comprises the testing for the lower verify level for the data state and the next program-verify operation comprises the testing for the higher verify level for the data state such that the control circuit is configured to perform program-verify between doses of programming by:

ramping up voltage on the first select line in order to test for the higher verify level while performing the scan of the results of the testing for the lower verify level and without waiting for the scan to complete; and
ramping up the voltages on unselected word lines in order to test for the higher verify level while performing a scan of the results of the testing for the lower verify level and without waiting for the scan to complete.

4. The non-volatile storage apparatus of claim 1, wherein:
the control circuit is further configured to perform program-verify between doses of programming by testing whether memory cells connected to a selected word line have threshold voltages greater than a lower verify level for a data state and testing whether memory cells connected to the selected word line have threshold voltages greater than a higher verify level for the data state between doses of programming;

the previous program-verify operation comprises the testing whether memory cells connected to the selected word line have threshold voltages greater than the lower verify level for the data state;

37

the testing whether memory cells connected to the selected word line have threshold voltages greater than a lower verify level for the data state includes the scan of the results; and

the next program-verify operation comprises the testing whether memory cells connected to the selected word line have threshold voltages greater than the higher verify level for the data state.

5. The non-volatile storage apparatus of claim 4, wherein: the control circuit is further configured to slow down programming for memory cells having threshold voltages greater than the lower verify level and less than the higher verify level.

6. The non-volatile storage apparatus of claim 1, wherein: during program-verify between doses of programming, the control circuit is further configured to apply the step signal to control a voltage source connected to unselected word lines; and

the control circuit is configured to ramp up the voltages on unselected word lines for the next program-verify operation by using the voltage source to ramp up voltage on unselected word lines to an overdrive voltage while performing the scan of the results of the previous program-verify operation and without waiting for the scan to complete in response to the step signal so that voltage applied to the unselected word lines rises in steps.

7. The non-volatile storage apparatus of claim 6, wherein: the control circuit is further configured to perform program-verify between doses of programming by testing for a lower verify level for a data state and testing for a higher verify level for the data state;

the previous program-verify operation comprises the testing for the lower verify level for the data state and the next program-verify operation comprises the testing for the higher verify level for the data;

during program-verify between doses of programming, the control circuit is further configured to apply a verify compare voltage to a selected word line and sense memory cells connected to the selected word line; and

the control circuit is further configured to slow down programming for memory cells having threshold voltages greater than the lower verify level and less than the higher verify level.

8. The non-volatile storage apparatus of claim 1, wherein: the ramping up voltage on the first select line comprises ramping up voltage on the first select line to a sensing enable voltage while performing the scan of the results of the previous program-verify operation and without waiting for the scan to complete, the voltage on the first select line reaches the sensing enable voltage after the scan of the results of the previous program-verify operation completes; and

ramping up the voltages on unselected word lines comprises ramping up the unselected word lines to an overdrive voltage while performing the scan of the results of the previous program-verify operation and without waiting for the scan to complete, the voltages on the unselected word lines reach the overdrive voltage after the scan of the results of the previous program-verify operation completes.

9. The non-volatile storage apparatus of claim 1, wherein: the doses of programming are voltage pulses applied to a selected word line, the voltage pulses increase in voltage magnitude from pulse-to-pulse.

10. The non-volatile storage apparatus of claim 1, wherein:

38

the plurality of non-volatile memory cells are arranged in NAND strings; and

the plurality of word lines and the select line are connected to each of the NAND strings.

11. The non-volatile storage apparatus of claim 10, further comprising:

- a second select line connected to each of the NAND strings, the first select line is connected to each of the NAND strings at a first end of the NAND strings, the second select line is connected to each of the NAND strings at a second end of the NAND strings;
- a plurality of bit lines connected to the NAND strings at the first end of the NAND strings; and
- a source line connected to the NAND strings at the second end of the NAND strings, the control circuit is further configured to apply a verify compare voltage to a selected word line and sense memory cells connected to the selected word line as part of the performing program-verify between doses of programming.

12. A method, comprising:

- applying doses of programming to a set of non-volatile memory cells connected to a selected word line; and
- performing program-verify between the doses of programming including testing for a lower verify level for a data state and testing for a higher verify level for the data state, the testing for a higher verify level for the data state comprises:
 - ramping up voltages on a select line in order to test for the higher verify level while performing a scan of the results of the testing for the lower verify level and without waiting for the scan to complete; and
 - ramping up the voltages on unselected word lines in order to test for the higher verify level while performing the scan of the results of the testing for the lower verify level and without waiting for the scan to complete.

13. The method of claim 12, further comprising: slowing down programming in response to memory cells having threshold voltages greater than the lower verify level for the data state.

14. The method of claim 12, wherein the testing for the higher verify level further comprises:

- applying a step signal to control a voltage source connected to unselected word lines;
- the ramping up the voltages on unselected word lines comprises using the voltage source to ramp up voltage on unselected word lines to an overdrive voltage concurrently with, and without waiting for the completion of, the scan of the results of the testing for the lower verify level in response to the step signal so that voltage applied to the unselected word lines rises in steps.

15. The method of claim 12, wherein: the ramping up the voltages on unselected word lines is done in steps.

16. The method of claim 12, wherein the testing for the higher verify level for the data state further comprises:

- applying a verify compare voltage to a selected word line; and
- sensing memory cells connected to the selected word line.

17. The method of claim 12, wherein:

- the ramping up voltages on the select line raises the select line first to a positive supply voltage and then to a sensing enable voltage;
- the select line reaches the positive supply voltage before the scan of the results of the testing for the lower verify level completes; and

39

the select line reaches the sensing enable voltage after the scan of the results of the testing for the lower verify level completes.

18. The method of claim **12**, wherein:

the ramping up voltages on unselected word lines raises the unselected word lines to an overdrive voltage; the ramping up voltages on unselected word lines starts before the scan of the results of the testing for the lower verify level completes; and the unselected word lines reach the overdrive voltage after the scan of the results of the testing for the lower verify level completes.

19. A non-volatile storage apparatus, comprising:

a plurality of non-volatile memory cells;
a plurality of word lines connected to the memory cells;
a select line connected to the memory cells; and
a control circuit connected to the memory cells and word lines as well as the select line, the control circuit is configured to program the memory cells by applying a dose of programming and verify status of programming after applying the dose of programming, the verifying status of programming comprises verifying status for a lower verify level for a data state and verifying status for a higher verify level for the data state after applying the dose of programming, the verifying status for the lower verify level for the data state comprises a first sensing of memory cells to determine which memory

40

cells have threshold voltage higher than the lower verify level and scanning results of the first sensing, the verifying status for the higher verify level for the data state comprises:

ramping up voltage on a select line to a sensing enable voltage concurrently with, and without waiting for the completion of, the scanning results of the first sensing;

applying a step signal to control a voltage source connected to unselected word lines;

using the voltage source to ramp up voltage on unselected word lines to an overdrive voltage concurrently with, and without waiting for the completion of, the scanning results of the first sensing, in response to the step signal so that voltage applied to the unselected word lines rises in steps;

applying a verify compare voltage to a selected word line; and

sensing memory cells connected to the selected word line.

20. The non-volatile storage apparatus of claim **19**, wherein:

the plurality of non-volatile memory cells are arranged in NAND strings; and

the plurality of word lines and the select line are connected to each of the NAND strings.

* * * * *